SST-200 Back-Pressure Turbine Automation - Code Documentation

This documentation explains the code logic for the SST-200 turbine automation pipeline in a flow-wise format.
It is intended as a living document, so more sections can be added progressively for each flow path and sub-flow.

---

1) What this code does

This codebase automates core engineering tasks that are usually manual in SST-200 back-pressure turbine design:

- selecting and preparing Kreisl/DAT templates,
- generating and updating load points,
- launching Kreisl and Turba runs,
- checking ERG results,
- optimizing valve/nozzle behavior,
- validating final power match,
- producing HMBD-aligned output behavior (open/closed variants).

The objective is to reduce repetitive manual work, keep calculations consistent, and shorten turnaround time.

---

2) Main flow path first (top-level)

The code follows one top-level dispatch path, then branches into sub-flows:

1. **Main flow path (entry)**
   - `Program` -> `StartExec.Main4(...)` for standard path.
   - `MainExecutedClass.GotoBCD1120()` / `GotoBCD1190()` for executed path.
   - `CustomExecutedClass.Main_CustomFlowPathTest()` for custom path (direct or fallback).
2. **Standard flow path**
   - Base automation flow (`StartExec.Main4`) with template selection + Turba + ERG + valve + power match.
3. **Executed flow path**
   - Criteria-driven flow (`MainExecuted(criteria)`), where criteria can be:
     - `BCD1120`
     - `BCD1190`
     - `Throttle` (limited retry, then custom handoff)
4. **Custom flow path**
   - Heavy optimization flow (`Main_CustomFlowPathTest`) with custom DAT selection, PSO, custom ERG checks, valve optimization, and final power closure.
5. **Additional load points flow**
   - Activated in standard/executed/custom when customer load points exceed base set (`CustomerLoadPoints.Count > 1`), and iterated LP-wise.

For **overview diagrams of all four paths in one place**, see the Flowcharts gallery (all automation paths overview) below. Detailed subgraphs remain in **Sections 5–9**.

---

3) HMBD logic families used in code

A. Open HMBD

- Open cycle **with PST**
- Open cycle **without PST**

B. Closed HMBD

Closed-cycle handling is entered when:

- `DeaeratorOutletTemp > 0`

Inside this branch, it splits into:

1. **Dump condenser ON** (`DumpCondensor == true`)
2. **Dump condenser OFF** (`DumpCondensor == false`)

Then template choice is refined by:

- PRV feasibility check (`tsatvonp(ExhaustPressure * 0.92 - 0.25) - DeaeratorOutletTemp`),
- ERG presence (`File.Exists(KREISL.ERG)`),
- desuperheater decision (`exhaustTemp < PST`).

---

4) Code locations (quick map)

For **explicit execution entry methods** (`Main4` , `MainKreisL` , `MainExecuted` , custom, additional LP), see **Section 11: Code documentation — execution starting points**.

- `src/kreisl.cs`
  - `StartKreisl.MainKreisL(...)` orchestration
  - `FillInputValues()` branch behavior for open/closed conditions
- `src/core/Handlers/KreislDATHandler.cs`
  - `RefreshKreislDAT()` template selection and copy/update logic
- `src/core/HMBD/HMBD_Configuration.cs`
  - HMBD pre-feasibility and extraction behavior
- `src/core/Utilities/PrintPDF.cs`
  - closed-cycle output template combinations for dump/deaerator states

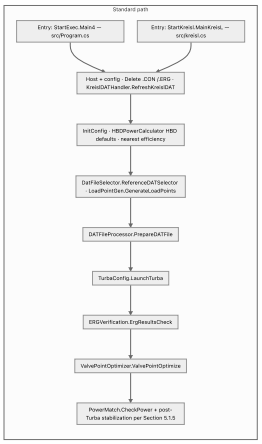
---

Flowcharts gallery (all automation paths overview)

This section collects **compact end-to-end flowcharts** for the four pipelines, plus **sub-charts** under each path that zoom into the main substeps (what actually runs between the big boxes). Full-size template and branch logic live in **Sections 5–9**.

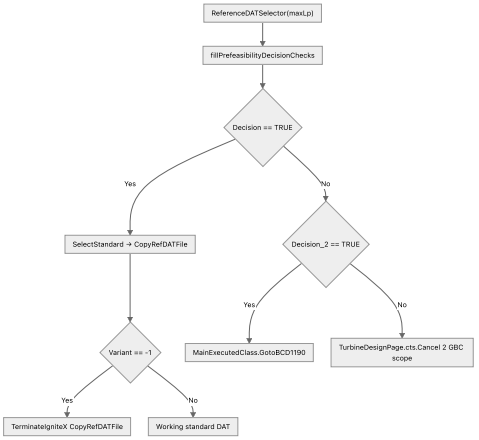
Standard path flowchart (overview)

Detailed template selection (`RefreshKreislDAT`), Kreisl launches, intermediate Turba/varicode rename, and bending/thrust escalation are documented in Section 5 and Section 6.

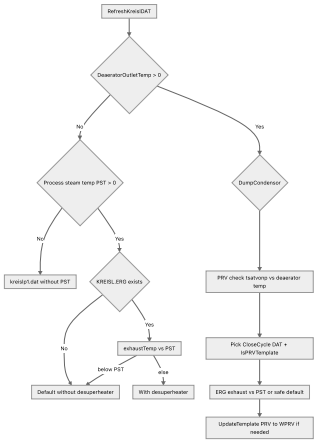


Standard path — sub-charts (what happens inside the boxes)

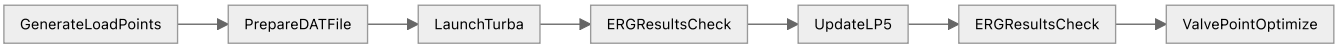
Sub-chart STD-A — Pre-feasibility and who picks the reference DAT (`DatFileSelector`)



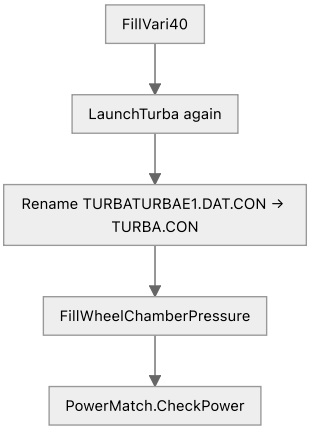
Sub-chart STD-B — Kreisl template family (`RefreshKreislDAT` logic, simplified)



Sub-chart STD-C — Compute core (between “LPs generated” and “valve optimize”)



Sub-chart STD-D — Closure strip (tie Turba ↔ Kreisl, then power)

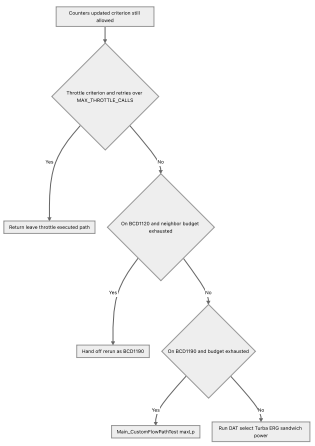


Executed path flowchart (overview)

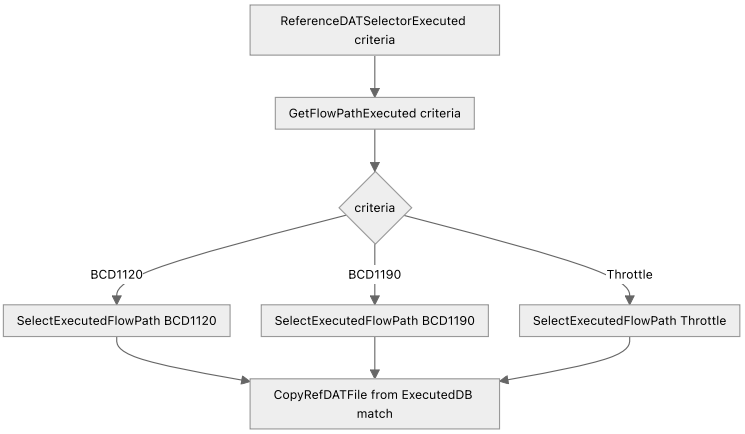
Criteria lifecycle, fallback `BCD1120 → BCD1190 → Main_CustomFlowPathTest`, and per-checker diagrams are in **Section 7**.

Executed path — sub-charts (criteria gates and ERG sandwich)

Sub-chart EXE-A — Retry / fallback ladder (same idea as `MainExecuted`)

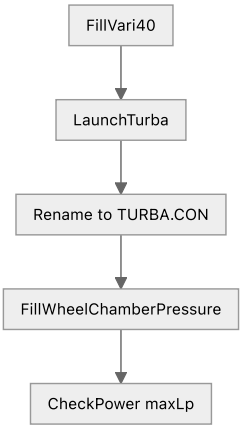


Sub-chart EXE-B — Reference DAT chosen by criterion



Sub-chart EXE-C — Turba + ERG “sandwich” around LP5

Sub-chart EXE-D — Final stabilization (same shape as standard tail)

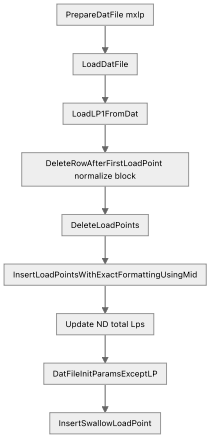


Custom path flowchart (overview)

Nearest DAT/PSO and `TurnaConvert` / `UpdatePunConvertor` detail are in Section 8.

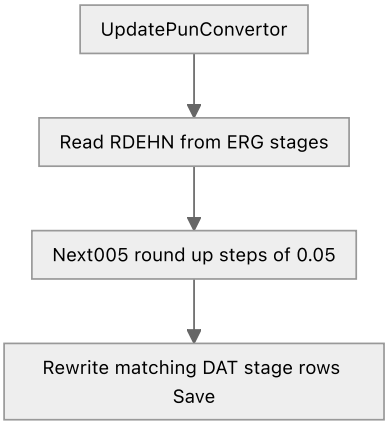
Custom path — sub-charts (DAT rebuild, PSO, ERG choice, closure)

Sub-chart CST-A — `PrepareDatFile(mxlp)` inside custom (DAT surgery)

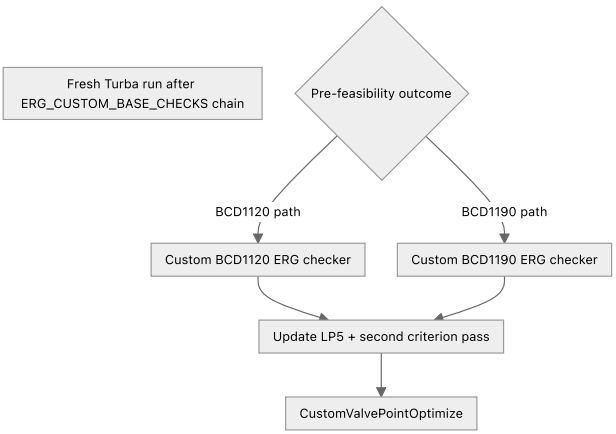


Sub-chart CST-B — After DAT is ready BCD rewrite + optimizer

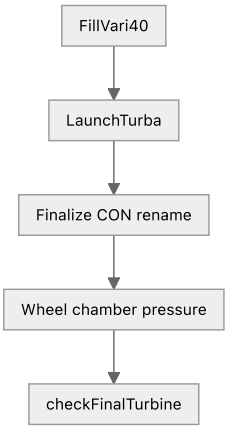
Sub-chart CST-C — ERG-derived feedback into DAT



Sub-chart CST-D — Criterion split after Turba restart

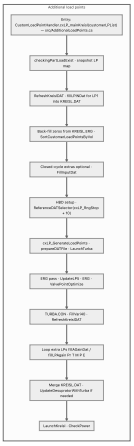


Sub-chart CST-E — Last mile to power



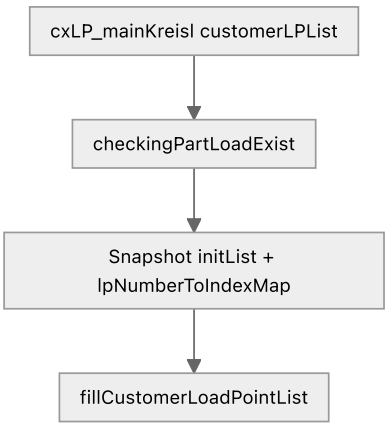
Additional load points flowchart (overview)

Symbols (`Pr/T/M/P/E`), `fillLPINDat` , `MainTemp` , and Kreisl merge loops are spelled out in Section 9.

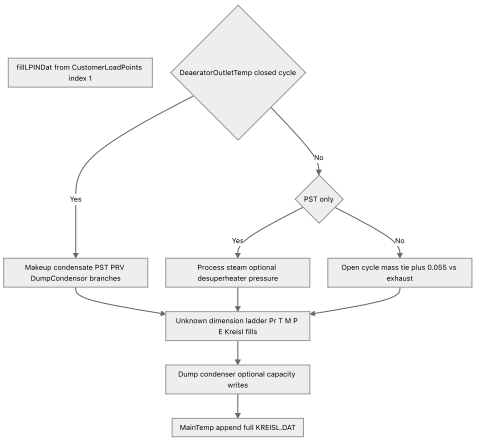


Additional load points — sub-charts (Kreisl LP1, mirrored standard block, per-LP merge)

Sub-chart ALP-A — Startup checks and snapshots

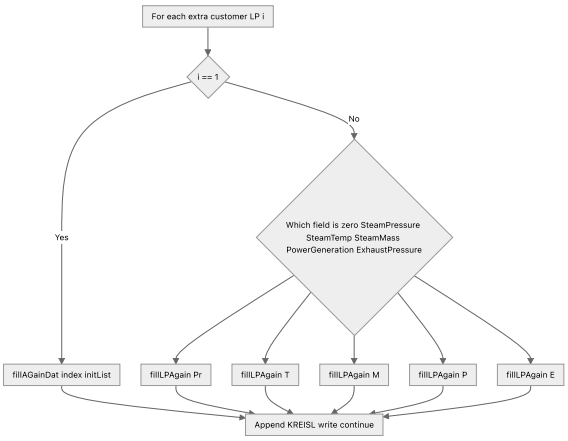


Sub-chart ALP-B — Kreisl LP1 shaping (`fillLPINDat`) at a glance

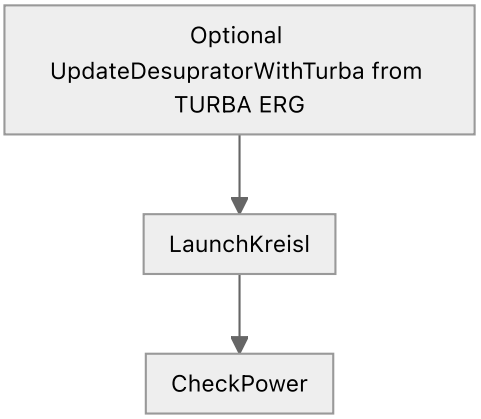


Sub-chart ALP-C — Mirror of standard pipeline at scaled LP count (middle block)

Sub-chart ALP-D — Loop over extra customer LPs (unknown dimension)



Sub-chart ALP-E — Closeout across cycles



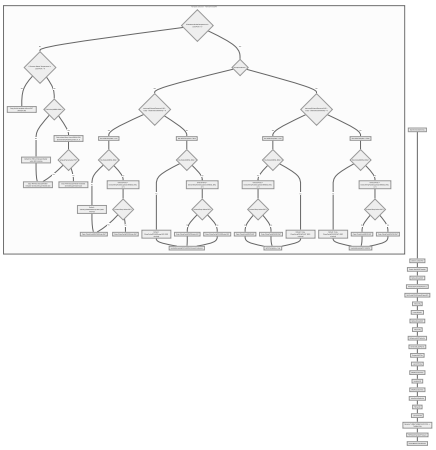
---

5) Flowchart - Standard path (starting section)

> **Overview chart:** Standard path flowchart in the flowcharts gallery.

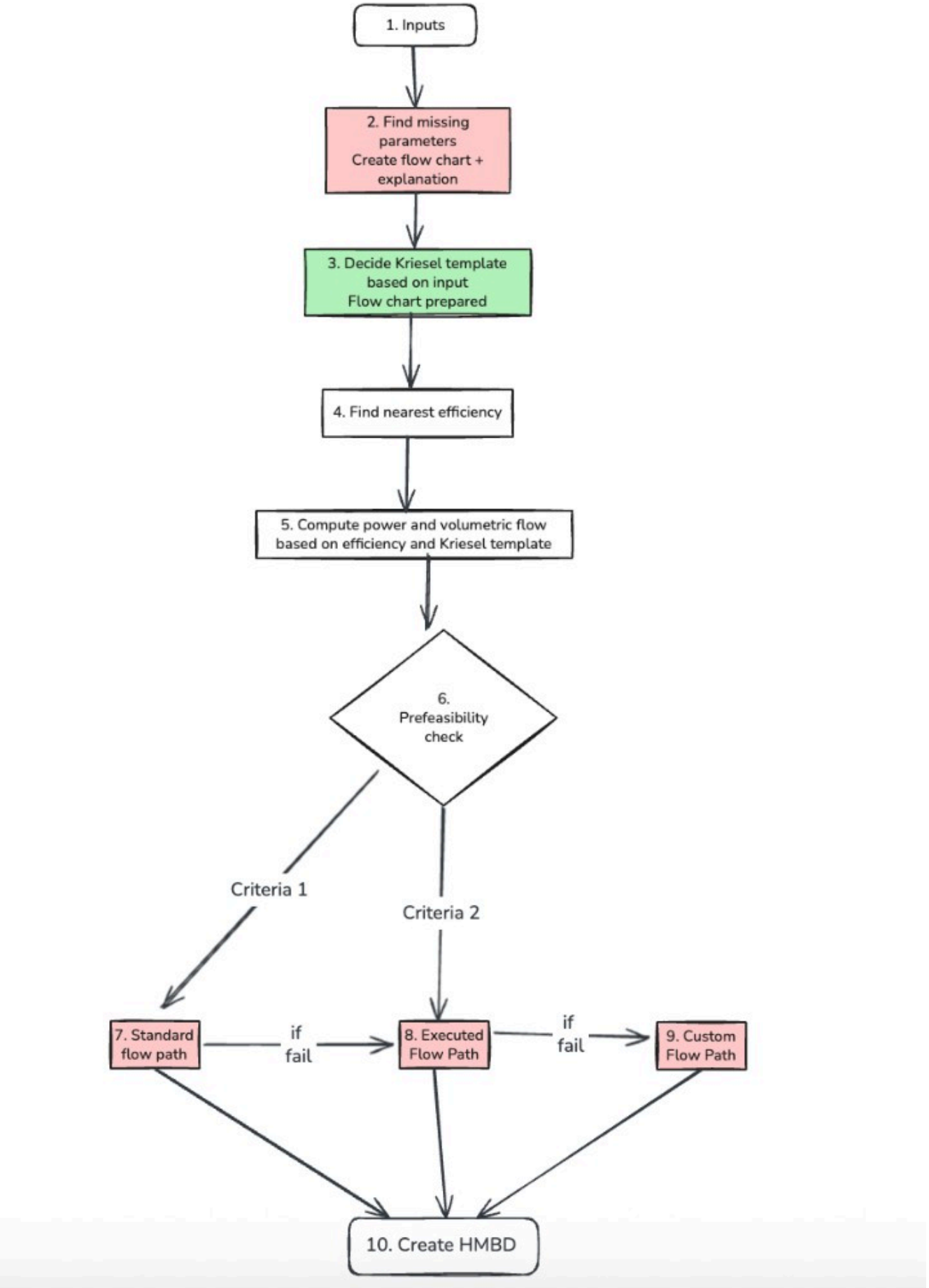
Below is the **detailed standard-path flowchart**, including Kreisl template selection (`RefreshKreislDAT`). Executed, custom, and additional-LP summaries are in the same gallery; **Sections 7–9** retain the full breakdowns.

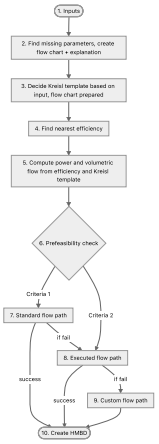
Reading the spine: the outer vertical chain uses node IDs `A` … `W` (letters only for Mermaid readability). `D` expands into subgraph **template selection** (inner diamonds `T1`, `T2`, … — different from `T2a`, which is the *second Turba launch* later on the spine). A full glossary of `A` – `W` is in Section 6.0: Letter legend.



5.0 End-to-end pipeline (shared diagram)

This is the clean top-level view of how inputs move through template selection, efficiency, prefeasibility, and the Standard → Executed → Custom fallback chain before **Create HMBD**. It sits before the detailed Standard-path splits in Section 5.1.

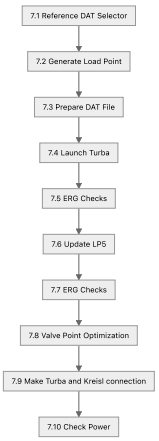

1. Inputs
2. Find missing parameters
Create flow chart + explanation
3. Decide Kriesel template based on input
Flow chart prepared
4. Find nearest efficiency
5. Compute power and volumetric flow based on efficiency and Kriesel template
6. Prefeasibility check
Criteria 1
Criteria 2
7. Standard flow path
if fail
8. Executed Flow Path
if fail
9. Custom Flow Path
10. Create HMBD



5.1 Standard flow path chart (clean split, as shared)

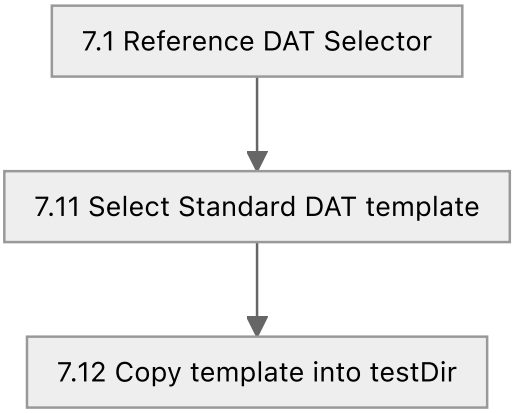
To keep the Standard section readable (not messy), the same logic is split into smaller charts exactly like your diagram style.

5.1.1 Main Standard pipeline (7.1 to 7.10)



This is the base Standard run sequence before deeper sub-logic.

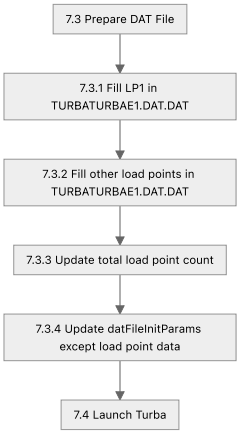
5.1.2 Reference DAT selector detail (7.1.x)



Explanation:

- `7.11` chooses the correct Standard template from input condition.
- `7.12` copies that template to runtime location (`testDir`) so later steps always work on the active DAT.

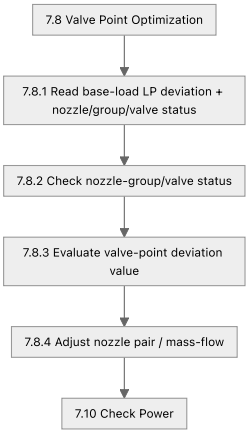
5.1.3 Prepare DAT detail (7.3.x)



Explanation:

- LP1 is written first, then remaining LPs are appended.
- Load-point count is synchronized with written rows.
- Non-LP init params are refreshed before Turba launch.

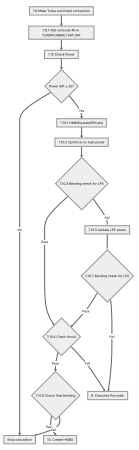
5.1.4 Valve optimization detail (7.8.x)



Explanation:

- This block reduces valve-point deviation iteratively.
- Based on status + deviation band, code applies nozzle pair/mass-flow corrections.
- Control then returns to `7.10 Check Power`.

5.1.5 Turba-Kreisl connection + power decision (7.9.1, 7.10.x)



Explanation:

- `7.9.1` adds the coupling marker (`varicode 40`) so Turba-Kreisl handoff is complete.
- `7.10` gates success by power difference first.
- If LP5/bending/thrust checks fail repeatedly, flow escalates to `8. Executed flow path`.
- If all checks pass, flow closes at `10. Create HMBD`.

---

6) Theory walkthrough - explain the full standard flowchart

This section explains what each major block in the flowchart is doing and why it exists.

6.0 Legend — Section 5 main-spine diagram letters (`A` … `W`)

The large **standard-path Mermaid diagram** at the start of Section 5: Flowchart — Standard path gives each step on the outer chain a short **node ID** (`A` , `B` , `C` , …). Those IDs exist **only inside that figure** — they are not C# identifiers.

When a subsection heading below writes **(Section 5: `X` → `Y`)**, it means "the part of Section 5's diagram from node `X` through node `Y` ."

Spine IDs and what they label in Section 5:

ID	Labels in Section 5 diagram
`A`	`StartKreisl.MainKreisL`
`B`	Delete `.CON` / `.ERG`
`C`	Create `KreislDATHandler`
`D`	`RefreshKreislDAT` (same box that leads into subgraph **template selection**; inner nodes there are named `T1` , `T2` , … — see Section 6.2)
`E`	`FillClosestTurbineEfficiency`
`F`	`GetTurbaCON(ClosestProjectID)`
`G`	`InitConfig` (after Kreisl setup)
`H`	`LaunchKreisL`
`I`	`RefreshKreislDAT` (second sync)
`J`	`InitConfig` (after second refresh)
`K`	`ReferenceDATSelector`
`L`	`GenerateLoadPoints`
`M`	`PrepareDATFile`
`N`	`LaunchTurba`
`O`	`ERGResultsCheck`
`P`	`UpdateLP5`
`Q`	`ERGResultsCheck` (second pass)
`R`	`ValvePointOptimize`
`S`	`FillVari40`
`T2a`	Second `LaunchTurba` (label in diagram is `T2a` so it does not clash with template diamonds `T1` , `T2` , … inside `RefreshKreislDAT`)

ID	Labels in Section 5 diagram
`U`	Rename `TURBATURBAE1.DAT.CON` → `TURBA.CON`
`V`	`FillWheelChamberPressure`
`W`	`PowerMatch.CheckPower`

Do not confuse: Section 6.2 heading **"subgraph (`T`)"** refers to the **whole template-selection subgraph** in Section 5 (nicknamed `T` in prose). That is unrelated to spine node `T2a` (second Turba run).

6.1 Entry and cleanup (Section 5: `A` → `D`)

The flow starts from `StartKreisl.MainKreisL`, then immediately performs runtime cleanup:

- delete stale `.CON` / `.ERG` files,
- create `KreislDATHandler`,
- call `RefreshKreislDAT`.

Why this matters:

- old simulation artifacts can pollute a new run,
- template selection must happen before running Kreisl/Turba,
- all later calculations depend on this initial DAT state.

6.2 Template selection subgraph (`T` = inner Section 5 subgraph on node `D`)

`RefreshKreislDAT` is the most important decision engine in the standard path. **In prose here, `T` means the nested "template selection" logic** hanging off `D` in Section 5 (`tmplSel`), not spine node `T2a`.

In the Section 5 Mermaid diagram, decision node `T1` asks: *"Is `DeaeratorOutletTemperature` in the load point greater than zero?"*

- **`T1 = Yes`** — you leave `T1` on the **Yes** arrow → **closed-cycle with deaerator** branch.
- **`T1 = No`** — you leave `T1` on the **No** arrow → **`DeaeratorOutletTemperature` is not greater than zero** (not set / zero) → treated as **open-cycle / PST** side of template selection ("no deaerator outlet temp").

So **`T1` is only a diagram shortcut for that first diamond**; it is not a separate variable in code.

It first determines whether the request is **closed-cycle-like** or **open-cycle-like**:

- if `DeaeratorOutletTemperature > 0` -> closed-cycle branch,
- else -> open-cycle/PST branch.

6.2.1 Open-cycle / PST side (`T1 = No`)

If no deaerator outlet temperature is provided:

- when `PST <= 0`, it copies `kreislp1.dat` (without PST path),
- when `PST > 0`, it tries to read `KREISL.ERG` and extract exhaust temperature.

Then it compares `exhaustTemp` with `PST`:

- `exhaustTemp < PST` -> choose **without desuperheater** template,
- otherwise -> choose **with desuperheater** template.

If ERG is missing, the flow safely defaults to the **without desuperheater** template.

6.2.2 Closed-cycle with deaerator (`T1 = Yes`)

When `DeaeratorOutletTemperature > 0`, the next split is dump condenser:

- `DumpCondensor == true` (dump ON),
- `DumpCondensor == false` (dump OFF).

Inside both dump ON/OFF paths, code checks PRV feasibility:

- `tsatvonp(ExhaustPressure*0.92 - 0.25) - DeaeratorOutletTemp > 0`.

That condition determines whether `IsPRVTemplate` stays true or false.

After that, it optionally reads `KREISL.ERG` and compares `exhaustTemp < PST` to decide:

- with-desuperheater template vs without-desuperheater template.

For non-PRV outcomes, the selected PRV template is converted using:

- `UpdateTemplatePRVToWPRVInDumpCondensor` (dump ON),
- `UpdateTemplatePRVToWPRV` (dump OFF).

This conversion step is essential because template families are reused and then adjusted to match final mode.

6.3 Post-template thermodynamic initialization (Section 5: `D` → `J`)

After template decision:

1. `FillClosestTurbineEfficiency` loads nearest known performance context.
2. `GetTurbaCON(ClosestProjectID)` binds reference project CON data.
3. `InitConfig` hydrates runtime model state.
4. `LaunchKreisL` runs Kreisl with selected template/input.
5. `RefreshKreislDAT` + `InitConfig` run again to sync generated outputs back into the pipeline.

The key idea is: **select -> run -> resync** before entering final DAT/Turba checks.

6.4 Main computation pipeline (Section 5: `K` → `R`)

This is the operational sequence:

1. `ReferenceDATSelector` picks final DAT reference.
2. `GenerateLoadPoints` builds LP inputs.
3. `PrepareDATFile` writes LP and control values into DAT.
4. `LaunchTurba` runs turbine simulation.
5. `ERGResultsCheck` validates result quality.
6. `UpdateLP5` modifies LP5 scenario and checks ERG again.
7. `ValvePointOptimize` adjusts valve configuration for convergence/performance.

Why LP5 is checked again:

- LP5 often acts as a corrective or boundary operating point,
- second ERG check ensures the updated point still satisfies constraints.

6.5 Final stabilization and power closure (Section 5: `S` → `T2a` → `U` → `V` → `W`)

This is the **tail of the Section 5 spine after valve optimization** — not only `S` and `W`, but every hop in between (see **Section 6.0**). Some older notes abbreviated this as "`S` → `W`"; the diagram's full chain is below.

After valve optimization:

1. **`S`** — `FillVari40` updates DAT/Kreisl variable settings.
2. **`T2a`** — `LaunchTurba` runs once more on updated values.
3. **`U`** — `Rename TURBATURBAE1.DAT.CON -> TURBA.CON` normalizes output naming for downstream use.
4. **`V`** — `FillWheelChamberPressure` pushes wheel chamber pressure back to Kreisl/DAT side.
5. **`W`** — `PowerMatch.CheckPower` performs final power closure.

This final block ensures the output is not just feasible, but also aligned with target power behavior.

6.6 Fallback and resilience behavior (conceptual)

Across this flow, the code uses practical fallback rules:

- if ERG file does not exist, choose safe default templates,
- if branch-specific PRV mode is not feasible, convert PRV templates to non-PRV variants,
- rerun key steps after major state updates (Kreisl run, LP update, valve optimization).

This makes the pipeline robust to missing intermediate files and branch-dependent state transitions.

---

7) Flow-wise content: Executed flow path

> **Overview chart:** Executed path flowchart in the flowcharts gallery.

7.1 Main executed flow (`MainExecutedClass.MainExecuted`)

The executed flow sequence is:

1. initialize counters and criteria limits (`mainCallCounters`, `throttleCounters`),
2. apply retry/fallback gates:
   - `Throttle` only up to `MAX_THROTTLE_CALLS`,
   - `BCD1120` exhaustion -> switch to `BCD1190`,
   - `BCD1190` exhaustion -> move to custom flow (`Main_CustomFlowPathTest`),
3. run HMBD defaults and nearest project selection (`PowerKNN`, `MoveYAndSetParams`),
4. select executed DAT (`ReferenceDATSelectorExecuted`),
5. load DAT and generate LPs (`LoadDatFile`, `GenerateLoadPoints`),
6. write DAT (`PrepareDATFileExecuted`),
7. validate wheel chamber pressure; if invalid, re-run executed selection,
8. launch Turba (`LaunchTurba`),
9. run ERG checks by criteria (`ErgResultsCheckExecuted(criteria, false)`),
10. update LP5 and re-check ERG (`UpdateLP5`, `ErgResultsCheckExecuted(criteria, true)`),
11. valve optimization (`ValvePointOptimize`),
12. final stabilization:
    - `FillVari40`
    - Turba re-launch
    - rename `TURBATURBAE1.DAT.CON` -> `TURBA.CON`
    - fill wheel chamber pressure
13. final power match (`CheckPower`).

7.2 Executed criteria sub-flows

- **BCD1120 flow**

- Uses `ErgResultsCheckBCD1120` in both initial and LP5-updated passes.
- If call budget is exhausted, auto-handoff to `BCD1190`.

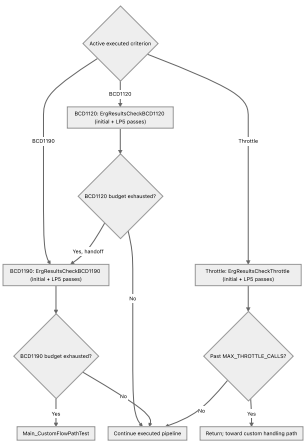
- **BCD1190 flow**
  - Uses `ErgResultsCheckBCD1190` in both initial and LP5-updated passes.
  - If call budget is exhausted, auto-handoff to custom flow.

- **Throttle flow**
  - Uses `ErgResultsCheckThrottle`.
  - Hard-limited retry count; beyond limit, flow returns and effectively shifts toward custom handling path.

7.3 Executed main pipeline flowchart

7.4 Executed criteria and ERG fallback flowchart

Each criterion uses the matching ERG checker on **both** passes in the main pipeline (`ErgResultsCheckExecuted(criteria, false)` then after `UpdateLP5`, `ErgResultsCheckExecuted(criteria, true)`). This chart shows how criteria **hand off** when budgets are exhausted.



7.5 `MainExecuted(criteria, maxLp)` in-depth flow and purpose

This is the **main controller** for the executed flow path.

In simple terms, it:

1. decides whether the current criterion is still allowed to run,
2. selects the nearest executed reference project,
3. rebuilds the DAT with executed load points,

4. runs Turba,
5. applies criterion-specific ERG checks,
6. updates LP5 and checks again,
7. optimizes valve behavior,
8. performs final stabilization and power matching,
9. falls back to the next path if the current executed path cannot close.

7.5.1 Easy overall picture

7.5.2 Counter and fallback logic

Before the executed calculation starts, `MainExecuted()` checks whether the current criterion is still allowed to continue.

- `mainCallCounters` tracks retries for `BCD1120` and `BCD1190`
- `throttleCounters` tracks retries for `Throttle`
- `MAX_THROTTLE_CALLS = 2`

Behavior:

- if the criterion is `Throttle` and the retry limit is exceeded, the method returns and effectively gives up on the throttle-executed path
- if `BCD1120` exceeds its allowed neighbor/call budget, the flow resets state and hands off to `BCD1190`
- if `BCD1190` exceeds its budget, the flow moves to `Main_CustomFlowPathTest(maxLp)`

So this method is not just a run pipeline. It is also the **gatekeeper for executed-path retry and fallback policy**.

7.5.3 Main executed setup phase

Once the criterion is accepted, the method performs the executed-run setup:

1. HMBD defaults are initialized:
   - `HBDsetDefaultCustomerParamas_Executed()` or Kreisl-specific variant
2. nearest executed candidates are resolved:
   - `PowerKNN(criteria)`
   - `MoveYAndSetParams()`
3. the executed DAT is selected:
   - `ReferenceDATSelectorExecuted(criteria)`
4. the selected DAT is loaded:
   - `LoadDatFile()`
5. executed load points are generated:
   - `GenerateLoadPoints(maxLp)`
6. turbine efficiency from the nearest match is pushed into runtime state:

- `HBDupdateEfficiency(efficiency)`

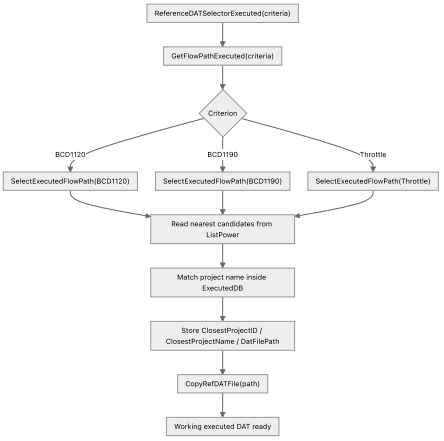
7. the DAT is rebuilt for the executed run:
   - `PrepareDATFileExecuted(maxLp)`

This means the executed flow first establishes the **nearest known project context**, then rewrites that selected DAT around the current request.

7.6 `ReferenceDATSelectorExecuted(criteria)` in-depth flow and purpose

This step is the executed-flow **reference project selector**.

It does not build a DAT from scratch. Instead, it chooses the closest executed reference project for the active criterion and copies that DAT into the working area.



What it really does:

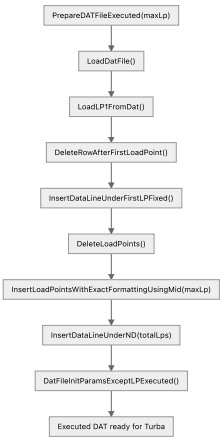
- `SelectExecutedFlowPath(criteria)` loops through the nearest candidates already prepared in `turbineDataModel.ListPower`
- it looks for entries marked with `KNearest == "Y"`
- it matches those candidate names against `ExecutedDB.ExecutedProjectDB`
- once it finds the matching project:
  - stores closest project metadata in `TurbineDataModel`
  - returns the DAT file path
- `CopyRefDATFile(path)` then copies that chosen executed DAT into the active runtime area

So this is the step that converts “nearest executed project” into an actual working DAT file.

7.7 `PrepareDATFileExecuted(maxLp)` in-depth flow and purpose

This method is the executed-flow **DAT reconstruction step**.

It rewrites the selected executed DAT with the generated executed load points and then refreshes executed-specific non-LP initialization values.



Important meaning:

- LP1 is preserved from the selected executed reference DAT
- old LP blocks are removed
- new executed LPs are inserted
- the ND/load-point count line is updated
- executed-specific DAT init parameters are refreshed after LP insertion

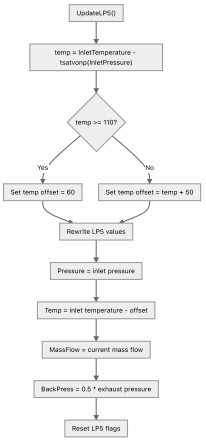
So this is the executed equivalent of the custom DAT rebuild step.

7.8 `UpdateLP5()` in-depth flow and purpose

This method regenerates the special LP5 case before the second ERG pass.

It uses current turbine inlet conditions and derives a corrective LP5 condition:

- pressure = inlet pressure
- temperature = inlet temperature minus a computed offset
- mass flow = current inlet mass flow
- back pressure = `0.5 * ExhaustPressure`
- RPM = LP1 RPM
- several flags (`InFlow` , `BYP` , `EIN` , `WANZ` , `RSMIN`) are reset



Why it matters:

- the first ERG check runs on the original executed LP set
- then LP5 is rebuilt into a stronger corrective/bending/thrust-sensitive case
- the second ERG check verifies whether the design still survives after that LP5 update

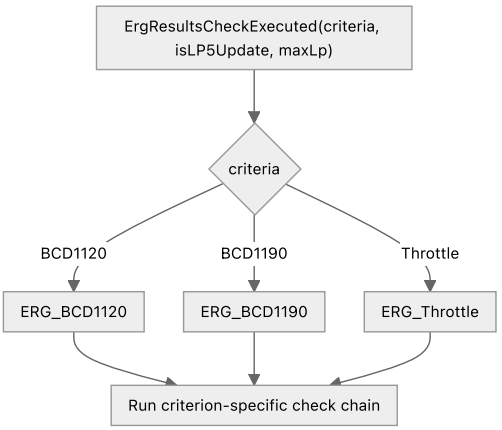
So `UpdateLP5()` is the executed flow's **second-pass stress/correction load-point generator**.

7.9 `ErgResultsCheckExecuted(criteria, isLP5Update, maxLp)` in-depth flow and purpose

This method is the **dispatcher** for executed ERG validation.

It does not perform the actual check logic itself. It routes to the criterion-specific checker:

- `BCD1120` -> `ErgResultsCheckBCD1120(isLP5Update, maxLp)`
- `BCD1190` -> `ErgResultsCheckBCD1190(isLP5Update, maxLp)`
- `Throttle` -> `ErgResultsCheckThrottle()`



7.9.1 `BCD1120` executed check chain

`ErgResultsCheckBCD1120(maxLp)` runs a staged validation chain:

1. exhaust volumetric-flow check
2. nozzle-section check
3. thrust-value check
4. Delta-T / GBC / wheel-chamber / PT / bending check
5. final `ErgResultsCheckBCD1120New(maxLp)` consolidation

Important fallback behavior:

- if nozzle optimization fails, it resets nearest-neighbor state and immediately hands off to `MainExecuted("BCD1190", maxLp)`

It also contains load-point repair behavior:

- if stage-pressure checks fail for MCR/high-BP points, it adjusts generated load points
- rewrites DAT with `PrepareDatFileOnlyLPUpdate()`
- reruns Turba
- re-enters the BCD1120 check flow

So BCD1120 is not a single yes/no check. It is a **repair-and-retry validation chain**.

7.9.2 `BCD1190` executed check chain

`ErgResultsCheckBCD1190(maxLp)` follows a similar pattern, but with 1190-specific limits:

1. exhaust check using delta-T-dependent exhaust curves
2. nozzle-section optimization check
3. thrust check
4. Delta-T / GBC / wheel-chamber / bending check
5. final `ErgResultsCheckBCD1190New(maxLp)` consolidation

Important fallback behavior:

- if exhaust or nozzle checks fail badly, it usually re-calls `MainExecuted("BCD1190", maxLp)` to try the next neighbor
- once executed retries are exhausted, the higher-level `MainExecuted()` logic escalates to the custom path

So BCD1190 is the **second executed rescue path** before custom flow is used.

7.9.3 `Throttle` executed check chain

`ERGResultsCheckThrottle()` runs its own sequence:

1. exhaust volumetric flow check
2. Delta-T / GBC / wheel-chamber / PT / bending check
3. thrust check
4. load-point pressure/power correction check
5. exhaust check

If load-point pressure checks fail:

- mass flow is increased for MCR points
- BP can be reduced for the high-BP case
- DAT is updated with `PrepareDATFileOnlyLPUpdate()`
- Turba is re-launched
- throttle ERG checks are re-run

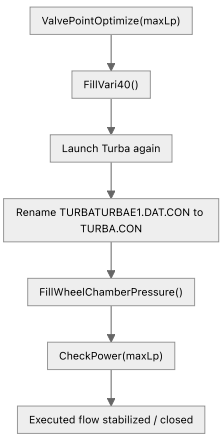
So throttle behaves like a smaller executed sub-flow with its own repair loop.

7.10 `ValvePointOptimize(maxLp)` and final stabilization

After both ERG passes complete, the executed flow enters the final executed stabilization block.

Sequence:

1. `ValvePointOptimize(maxLp)`
2. `FillVari40()`
3. Turba re-launch
4. normalize output CON file to `TURBA.CON`
5. push wheel chamber pressure back into Kreisl/DAT side
6. `CheckPower(maxLp)`



Meaning:

- valve-point optimization tries to improve convergence/performance before final closure
- `Vari40` reconnects Turba and Kreisl state

- the wheel chamber pressure is pushed back into the Kreisl-side files
- `CheckPower()` is the final gate for executed success

7.10.0 Inside `ExecPowerMatch.CheckPower(maxLp)` (executed) — full closure flow

`CheckPower(maxLp)` lives in `src/core/Checks/Exec_ERG_PowerMatch.cs` (class `ExecPowerMatch`). It is the **last "decision gate"** of the executed path: it tries to **close** on power, no-load, bending (LP5 repair), thrust, and "final bending across all LPs". If it cannot close within the internal budgets, it **falls back** to the custom path (`Main_CustomFlowPathTest`).

What it uses (inputs):

- **From Turba**: `TurbaOutputModel.OutputDataList[...]` (LP1 power/efficiency, LP5 bending + thrust, etc.)
- **From Kreisl/HBD**: `turbineDataModel.FinalPower` (target power), plus HMBD configuration defaults
- **From file system**:
  - `C:\testDir\AdminControl.csv` → `PowerMargin` (default 25 if missing)
  - `C:\testDir\TURBATURBAE1.DAT.DAT` for generator/turbine/gearbox spec extraction + "AUS..." soft-check parameter edits

High-level phases in order (as implemented):

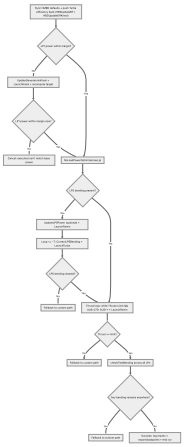
1. **Sync HMBD defaults and update efficiency**
   - `ExecHMBDConfiguration.HBDSetDefaultCustomerParams()`
   - reads **Efficiency** from Turba (`OutputDataList[1].Efficiency`)
   - writes the efficiency back:
     - if `StartKreisl.kreislKey`: `HBDUpdateEffKriesl(Efficiency, maxLp)` (Kreisl+Turba coupling rounds)
     - else: `HBDupdateEff(Efficiency)`
2. **Power match on LP1 vs HBD target**
   - target power = `floor(turbineDataModel.FinalPower)` (assigned into `turbineDataModel.AK25`)
   - compares against Turba LP1: `PowerLP1 = OutputDataList[1].Power_KW`
   - considers "matched" if:
     - `abs(targetPower - PowerLP1) <= getPowerMargin()` OR `targetPower <= PowerLP1`
   - if not matched, it tries one "generator sync" round:
     - `UpdateGeneratorinKriesl()` reads **generator/gearbox/turbine** powertrain specs from `TURBATURBAE1.DAT.DAT` and pushes them into Kreisl via `KreislDATHandler.updateGeneratorSpecs(...)`, `updateGearBoxPower(...)`, `updateTurbinePower(...)`
     - `KreislIntegration.LaunchKreisL()` then re-extracts `AK25` from ERG
     - if still not matched: logs failure and cancels
3. **No-load optimization**
   - `NoLoadPowerOptimize(maxLp)` delegates to `ExecNoLoadPowerOptimizer.NoLoadPowerOptimize(maxLp)`
4. **LP5 bending loop (repair + re-run Turba)**
   - reads bending from Turba: `OutputDataList[5].Bending`
   - if bending exists:
     - `UpdateLP5Power(maxLp)` modifies Kreisl LP5 template inputs and runs Kreisl+Turba to refresh the LP5 operating point
     - sets `isLP5Change = true`
   - then `TurbaAutomation.LaunchRsmin()` and starts a bounded loop:
     - while LP5 bending still exists, up to **~7 iterations**:
       - `CorrectLP5Bending()` patches the Turba DAT blade table (see **Section 7.10.1**)
       - `TurbaAutomation.LaunchTurba(maxLp)` reruns Turba so the next ERG reflects the patch

- if bending still exists after the loop:
  - falls back to custom: `CustomExecutedClass.Main_CustomFlowPathTest(maxLp)` and returns

5. **Thrust loop (soft-check adjust "AUS..." parameter)**
   - while `LP5.Thrust > ThrustLimit` **and** `getAUS() > 270`:
     - `getAUS()` reads the value under the DAT label `! AUSGLEICHSKOLBENDURCHMESSER`
     - `UpdateDATSoftChecks(getAUS() + 1)` increments this parameter in `TURBATURBAE1.DAT.DAT`
     - reruns `LaunchRsmin()` to re-evaluate thrust with the new soft-check value
   - if thrust is still above limit after the loop:
     - falls back to custom: `Main_CustomFlowPathTest(maxLp)` and returns

6. **Final bending check across all load points**
   - `checkFinalBending(maxLp)` scans all LPs (1..maxLoadPoints-1) and fails if **any** `OutputDataList[lp].Bending` is non-empty
   - if passed:
     - runs `LaunchRsmin()`, `UpdateOutletTempAndEnth()`, logs "turbine is good", writes final power/efficiency, writes/load-points outputs, then cancels `finalToken` to end the run
   - if failed:
     - falls back to custom path again (then cancels)

Mini flow (executed `CheckPower`):



7.10.1 Inside `CheckPower(maxLp)`: `CorrectLP5Bending()` (executed) — easy purpose + flow

`CorrectLP5Bending()` lives in `src/core/Checks/Exec_ERG_PowerMatch.cs` (class `ExecPowerMatch`).

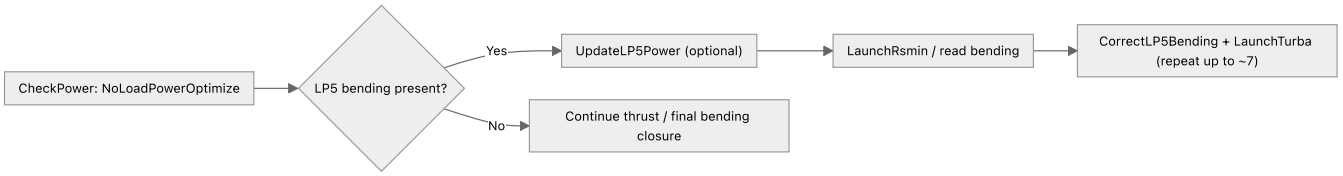
What it is for (in simple words):

- This is part of the `CheckPower(maxLp)` **closure loop**. After no-load optimization, `CheckPower` looks at LP5 bending; if bending is reported it keeps trying repairs and re-running Turba.
- LP5 is used as a "stress / correction" operating point (see **Section 7.8 UpdateLP5**).
- After Turba runs, the ERG contains stage-wise **bending status flags** for LP5.
- If any stage in LP5 reports a bending flag (`F` or `B`), `CorrectLP5Bending()` **patches the blade table in the Turba DAT** (`TURBATURBAE1.DAT.DAT`) for the affected stage row(s), so the next Turba run is more likely to pass bending constraints.

Inputs / outputs:

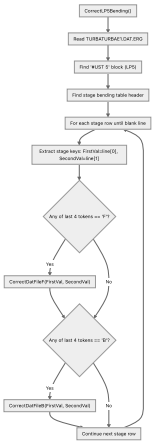
- **Reads**: `C:\testDir\TURBATURBAE1.DAT.ERG`
- **Writes**: `C:\testDir\TURBATURBAE1.DAT.DAT` (via `CorrectDatFileF(...)` / `CorrectDatFileB(...)`)

Where it sits in the `CheckPower` loop (very high-level):



How it decides what to fix:

- It finds the LP5 block (`#UST 5`) in the ERG.
- It finds the stage table header line:
  - `STUFE SIGZV SIGAZS SIGAZF SIGVS SIGAS  GSS  HSS SIGVF SIGAF  HSF PRESZ PRESS GEF`
- It scans each stage row until a blank line.
- It looks at the **last 4 tokens** of the row; if any of those are:
  - `F` → call `CorrectDatFileF(stage, gi)`
  - `B` → call `CorrectDatFileB(stage, gi)`



What the DAT patchers do (detailed, with examples):

Both patchers are **small deterministic edits** to the `!ST` **blade table** in `TURBATURBAE1.DAT.DAT`. They keep one main invariant: the pair of coupled numbers (roughly " `SE` " and a linked count/setting next to it) is kept consistent by preserving the product ratio used by the code.

> In executed `Exec_ERG_PowerMatch.cs` rows are split by **spaces**. In custom `Cu_ERG_PowerMatch.cs` rows are split by `|` . The math is the same, but the field indices differ.

A) `CorrectDatFileF(stage, gi)` — enforce a minimum `SE` (= 25) + set correction mode

Trigger: any LP5 stage row has a bending token `F` (from ERG).

What it changes (executed):

- Find the matching `!ST ...` row where:
  - `lineArray[0] == stage` and `lineArray[1] == gi`
- Treat:
  - `SE = lineArray[4]`
  - `SZ = lineArray[5]`
  - "correction mode flag" = `lineArray[10]`
- If `SE < 25` then:
  i. compute: `ans = int64(SE * SZ / 25)` (floor/truncate)
  ii. set `SE = 25`
  iii. force `SZ` parity based on `gi` parity:
    - if `gi` is even → make `SZ` **odd**
    - if `gi` is odd → make `SZ` **even**

        - (done by incrementing `ans` by 1 when needed)
    iv. set correction flag `lineArray[10] = 2`
- Else (already `SE >= 25`):
    - only sets correction flag `lineArray[10] = 2`

Worked example (executed):

Assume a blade row for `(stage=3, gi=2)` has:

- `SE = 20`
- `SZ = 100`
- `gi = 2` (even)

The patch does:

- `ans = floor(20 * 100 / 25) = floor(80) = 80`
- `gi` is even ⇒ `SZ` must be **odd** ⇒ `SZ = 81`
- Set `SE = 25`
- Set correction flag field to `2`

So the pair goes from `(SE,SZ) = (20,100)` to approximately `(25,81)` while keeping the ratio logic close (since 20×100 ≈ 25×80), then nudged to 81 to satisfy parity rule).

Mini before/after (executed `!ST` row fields only):

Field (executed)	Before	After
`SE` (`lineArray[4]`)	20	25
`SZ` (`lineArray[5]`)	100	81
correction flag (`lineArray[10]`)	*(unchanged / whatever it was)*	2

B) `CorrectDatFileB(stage, gi)` — bump `SE` to the next allowed NB step + keep parity

Trigger: any LP5 stage row has a bending token `B` (from ERG).

What it changes (executed):

- For the matching `(stage,gi)` row, read:
    - `SE = lineArray[4]`
    - `SZ = lineArray[5]`
- Compute the next allowed step:
    - `SENextNB = getNextNB(SE)` where `getNextNB` scans the static `data` table's first column and picks the **next higher** value.
- Preserve the ratio with the new SE:
    i. `ans = floor(SE * SZ / SENextNB)`
    ii. set `SE = SENextNB`
    iii. enforce parity on `SZ` based on `gi`:
        - if `gi` is even → make `SZ` **odd**
        - if `gi` is odd → make `SZ` **even**
        - (again: bump `ans` by 1 when needed)

Worked example (executed):

Assume `(stage=4, gi=1)` has:

- `SE = 32.0`

- `SZ = 101`
- `gi = 1` (odd)

From the static table, the next higher `SE` after 32.0 is **40.0**.

- `ans = floor(32.0 * 101 / 40.0) = floor(80.8) = 80`
- `gi` is odd ⇒ `SZ` must be **even** ⇒ `SZ = 80` (already even, so keep)
- Set `SE = 40.0`

So the pair moves from `(32.0,101)` to `(40.0,80)` preserving the same "capacity" scale approximately and respecting the row parity rule.

Mini before/after (executed `!ST` row fields only):

Field (executed)	Before	After
`SE` (`lineArray[4]`)	32.0	40.0
`SZ` (`lineArray[5]`)	101	80

C) How this relates to bending repair

`CorrectLP5Bending()` does not guess new geometry randomly. It uses a **rule-based, repeatable edit**:

- `F` → ensure a hard minimum threshold (`SE >= 25`) + mark correction mode
- `B` → move `SE` to the next discretized "NB" step

Then `CheckPower` re-runs Turba and checks whether LP5 bending clears (looping up to ~7 iterations in both executed and custom).

7.11 Additional load points in executed flow

If the customer has more than two load points, the executed flow continues after power match into an additional-load-point merge path.

That block:

1. ensures `TURBA.CON` exists,
2. refreshes Kreisl DAT,
3. writes wheel chamber pressure back,
4. loops over extra customer LPs,
5. regenerates those LPs into `KREISL.DAT`,
6. launches Kreisl again on the merged file.

So the executed flow can end either as:

- a normal executed single/base LP solution, or
- an executed base solution followed by an additional-LP Kreisl expansion step.

---

8) Flow-wise content: Custom flow path

> **Overview chart:** Custom path flowchart in the flowcharts gallery.

8.1 Main custom flow (`CustomExecutedClass.Main_CustomFlowPathTest`)

The custom flow sequence is:

1. initialize dependencies and cleanup (`DeleteCONFiles` , `RefreshKreislDAT`),
2. read nearest Turba context and fill input values,
3. set HMBD defaults + initial efficiency setup,
4. generate custom load points (`CustomLoadPointGenerator.GenerateLoadPoints`),
5. pre-feasibility checks (`fillPrefeasibilityDecisionChecks`),
6. pick nearest custom params and custom reference DAT:
    - `GetNearestParams_Custom`
    - delete executed DAT
    - copy custom reference DAT,
7. prepare DAT (`CustomDATFileProcessor.PrepareDatFile`),
8. run base update and optimization:
    - `BCD_UPDATE`
    - PSO flow optimizer (`InvokeTurbineDesigner`),
9. run custom base checks (`ERG_CUSTOM_BASE_CHECKS`),
10. convert/update steam path (`TurnaConvert` , `UpdatePunConvertor`) and launch Turba,
11. select ERG criterion from pre-feasibility decision:

- custom BCD1120 check or
- custom BCD1190 check,

12. LP5 update + second criterion check,
13. custom valve point optimization (`CustomValvePointOptimizer`),
14. final closure:
    - `FillVari40`
    - Turba launch
    - rename/load final CON
    - fill wheel chamber pressure
    - custom power match + final checks (`checkFinalTurbine`).

8.2 Custom flow path flowchart

8.3 `GetNearestParams_Custom()` flow and purpose

This method is the custom **reference-DAT selection and parameter-seeding** step used in `Main_Custom.cs` .

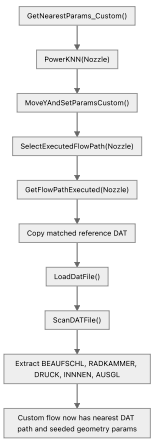
What it does:

1. runs a nearest-neighbor search for custom/nozzle candidates,
2. updates HMBD custom parameters from the chosen nearest project,
3. resolves and copies the matching reference DAT,
4. loads that DAT into memory,

5. scans the DAT and extracts the key geometry constants used later in the custom flow.

The extracted values are:

- `BEAUFSCHL`
- `RADKAMMER`
- `DRUCK`
- `INNNEN`
- `AUSGL`



Implementation breakdown:

- `PowerKNN("Nozzle")` filters the nearest-project search to nozzle-like candidates and stores the closest matches in `ListPower`.
- `MoveYAndSetParamsCustom()` moves/selects the active custom neighbor row and updates HMBD custom parameters when a valid row is found.
- `SelectExecutedFlowPath("Nozzle")` resolves the DAT path of the nearest matching project from the executed-project database.
- `GetFlowPathExecuted("Nozzle")` performs the actual copy of that reference DAT into the working area.
- `LoadDatFile()` reads `TURBATURBAE1.DAT.DAT` into `turbineDataModel.DAT_DATA`.
- `ScanDATFile()` parses the copied DAT and fills turbine constants used later by the custom flow.

This means `GetNearestParams_Custom()` is not doing optimization itself. It is preparing the **best starting reference DAT and initial custom geometry parameters** before later stages like custom DAT preparation, `BCD_UPDATE`, and PSO optimization begin.

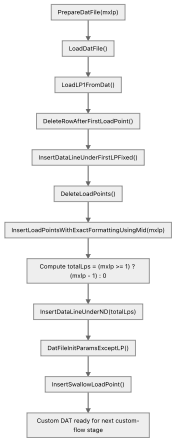
8.4 `CustomDATFileProcessor.PrepareDatFile(mxlp)` flow and purpose

This method is the custom-flow **DAT file preparation step** called from `Main_Custom.cs`.

Its job is to rebuild the working DAT file so Turba/custom-flow stages run on the correct load-point structure and updated initialization values.

What it does:

1. loads the active working DAT into memory,
2. reads the first load point already present in the DAT,
3. removes old LP rows / resets the LP area layout,
4. inserts the regenerated custom load points up to `mxlp`,
5. updates the ND/load-point count line,
6. refreshes DAT initialization parameters except LP data,
7. inserts the swallow load point used later by the run.



Implementation breakdown:

- `LoadDatFile()` reads `TURBATURBAE1.DAT.DAT` into `turbineDataModel.DAT_DATA`.
- `LoadLP1FromDat()` captures the original first load-point structure so the regenerated DAT keeps the expected LP1 baseline.
- `DeleteRowAfterFirstLoadPoint()` and `InsertDataLineUnderFirstLPFixed()` normalize the DAT block immediately under LP1 before bulk LP insertion.
- `DeleteLoadPoints()` clears previously existing load-point entries from the working DAT.
- `InsertLoadPointsWithExactFormattingUsingMid(mxlp)` writes the regenerated custom LP rows with the exact DAT formatting expected by downstream tools.
- `InsertDataLineUnderND(totalLps)` updates the ND section with the effective number of generated load points.
- `DatFileInitParamsExceptLP()` refreshes non-load-point initialization / machine parameters after the LP rewrite.
- `InsertSwallowLoadPoint()` appends the swallow operating point needed for later custom processing.

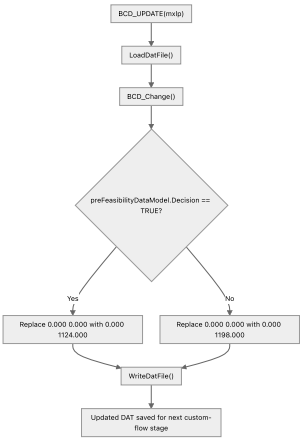
So `PrepareDatFile(mxlp)` is not selecting projects or optimizing anything by itself. It is the **DAT reconstruction step** that converts the chosen custom inputs and generated LPs into the final runtime DAT structure used by the next stages.

8.5 `customSaxaSaxi.BCD_UPDATE(mxlp)` flow and purpose

This method is the custom-flow **BCD rewrite step** used after DAT preparation.

Its role is simple but important: it loads the current DAT, updates the BCD-related value in the DAT based on the pre-feasibility decision, and writes the modified DAT back to disk.

At the current implementation level, the `mxlp` argument is passed in from `Main_Custom.cs`, but the actual `BCD_UPDATE()` method does not use it internally.



Implementation breakdown:

- `LoadDatFile()` reloads the current working DAT into memory.

- `BCD_Change()` scans for the `!  ABSTAND AXIALLAGER` section and checks the line immediately below it.
- If the next line starts with `0.000  0.000`, the code rewrites that line using the pre-feasibility result:
  - decision `TRUE` -> set BCD to `1124.000`
  - otherwise -> set BCD to `1198.000`
- `WriteDatFile()` saves the modified DAT back to `TURBATURBAE1.DAT.DAT`.

So `BCD_UPDATE(mxlp)` is not a full optimization stage by itself. It is a **targeted DAT parameter patch** that converts the custom-flow pre-feasibility decision into the correct BCD setting before downstream checks and optimizers run.

8.6 `pSOFlowPathOptimizerNozzle.InvokeTurbineDesigner()` in-depth flow and purpose

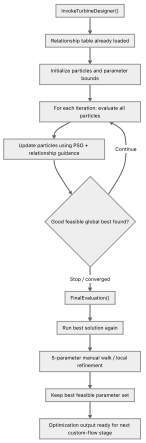
This is the **main custom nozzle optimization function** in the custom flow. The current implementation uses **relationship-aware PSO only** (`InvokeTurbineDesigner` → `PSOLoop` in `Cu_PSOFlowPathOptimizerNozzle.cs`).

In simple terms, it tries many combinations of five DAT parameters, runs Turba for each combination, rejects combinations that violate engineering rules, keeps the best feasible one, and then does a final refinement pass.

The five optimized parameters are:

- `B` = `BEAUFSCHL` (admission factor)
- `R` = `RADKAMMER` (wheel chamber pressure)
- `D` = `DRUCKZIFFERN` (stage-related setting, stored as negative in DAT)
- `I` = `INNENDURCHMESSER` (shaft diameter)
- `A` = `AUSGLEICHSKOLBEN` (balance piston diameter)

8.6.1 Easy overall picture



8.6.2 What this function is doing, step by step

1. **Start the relationship-aware PSO path**
   - `InvokeTurbineDesigner()` runs **only** this path: it logs the loaded engineering relationships and calls **`PSOLoop()`** (see `Cu_PSOFlowPathOptimizerNozzle.cs`). There is no alternate “LLM-guided” branch in this entry point.
2. **Initialize optimization search space**
   - `InitializeParameterBounds()` creates min/max/step values for `B, R, D, I, A`.
   - Bounds depend partly on process inputs like inlet pressure and backpressure.
   - Example:
     - `B` gets an admission-factor range,
     - `R` gets a wheel-chamber-pressure range,

- `D` gets a stage-related range,
- `I` and `A` get shaft/piston diameter ranges.

3. **Create initial particles**

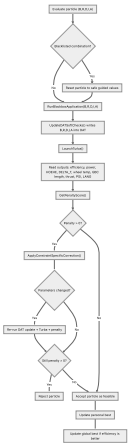
- `InitializeParticles()` creates the PSO population.
- Each particle is one candidate parameter set: `(B, R, D, I, A)`.
- `InitializeParticleWithRelationshipGuidance()` does not randomize blindly:
  - it starts from conservative values,
  - it biases values using engineering relationships,
  - for example smaller shaft + larger piston is preferred for thrust balance.

4. **Run the PSO loop**

- `PSOLoop()` currently runs a fixed number of iterations.
- In each iteration:
  - `EvaluateAndUpdateParticles()`
  - `UpdateParticlesWithRelationships()`
- Then it checks whether a strong feasible global best has already been found.

8.6.3 Detailed particle evaluation flow

This is the core of the optimizer because every particle is tested by actually updating the DAT and launching Turba.



8.6.4 What `RunBlackboxApplication()` means

This is the real “test a candidate” step:

1. `UpdateDATSoftChecks(B,R,D,I,A)` writes the five candidate values into the nozzle DAT file.
2. `LaunchTurba()` runs Turba on that updated DAT.
3. The code then reads the resulting outputs from `turbaOutputModel`.

So the optimizer is **not using a formula-only estimate**. It is using the actual Turba run as the black-box evaluator.

8.6.5 How penalty-based feasibility works

After each Turba run, `GetPenaltyScore()` is called.

- If penalty is `0`, the candidate is treated as **feasible**.
- If penalty is greater than `0`, the candidate violates one or more engineering constraints.

The checks include output conditions such as:

- nozzle height (`HOEHE`)

- nozzle area (`FMIN1`)
- wheel chamber temperature
- `DELTA_T`
- `GBC_Length`
- `PSI`
- `LANG`
- thrust per load point

The exact limits depend on whether the pre-feasibility branch implies **BCD1120** or **BCD1190**.

8.6.6 How correction works when a particle fails

If a particle is infeasible, the optimizer does **targeted correction** instead of discarding it immediately.

`ApplyConstraintSpecificCorrection()`:

1. detects whether the active check type is `1120` or `1190`,
2. reads current Turba outputs,
3. changes only the parameters that are most relevant to the failed constraint,
4. snaps the result back to valid step sizes.

Examples of correction logic:

- if nozzle area is too high -> reduce `B`
- if wheel chamber temperature is too high -> reduce `R`
- if `DELTA_T` is too high -> reduce `R`
- if GBC length is too large -> adjust `D`
- if PSI fails -> adjust stages (`D`)
- if thrust fails -> adjust shaft diameter `I`

After correction, the optimizer re-runs Turba and re-checks the penalty.

If the particle is still infeasible, it is rejected.

8.6.7 How PSO updates the particles

After evaluation, `UpdateParticlesWithRelationships()` moves particles for the next iteration.

This stage combines:

- normal PSO behavior:
  - current position
  - velocity
  - personal best
  - global best
- engineering relationship guidance:
  - shaft diameter vs thrust
  - piston diameter vs thrust
  - admission factor vs nozzle area
  - pressure vs wheel chamber temperature

So this optimizer is not a plain random PSO. It is a **relationship-aware PSO** that tries to move particles in directions that make engineering sense.

8.6.8 What happens if no good solution is found in an iteration

If no feasible particles are found:

- `ApplyEmergencyDiversification()` resets particles into broader but still sensible ranges.

There is also extra diversification support for poor-performing particles through:

- `AnalyzeRelationshipPerformance()`
- `ApplyRelationshipGuidedDiversification()`

These are meant to push the search away from bad regions and back toward more promising combinations.

8.6.9 Final evaluation and manual walk

After the PSO loop finishes, the code does **more than just accept the best PSO particle**.

`FinalEvaluation()`:

1. runs the current global best again,
2. checks final penalty and efficiency,
3. logs the best `B, R, D, I, A`,
4. performs a **manual walk** across each of the five parameters one by one,
5. keeps any improved feasible result.

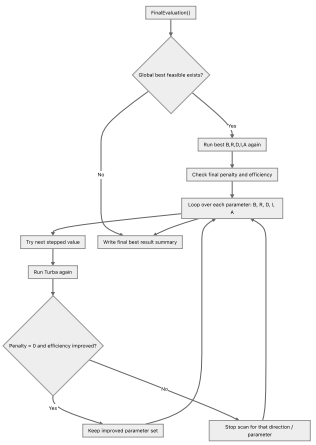
That manual walk is important because:

- PSO finds a strong candidate region,
- the final sweep then tries to squeeze out a little more efficiency,
- but only while keeping penalty at zero.

So the real flow is:

- broad guided search first,
- exact local improvement second.

8.6.10 Final refinement flow



8.6.11 Easy summary

`InvokeTurbineDesigner()` is the main engine that:

1. chooses a candidate nozzle DAT parameter set,
2. writes those values into the DAT,
3. launches Turba,
4. checks whether the result is feasible,
5. corrects bad candidates,
6. keeps the best feasible solution,

7. finally refines that solution again before handing it back to the custom flow.

So in one sentence: this function is the **main black-box optimizer that searches for the best feasible custom nozzle design by repeatedly editing the DAT, running Turba, enforcing engineering constraints, and refining the best result**.

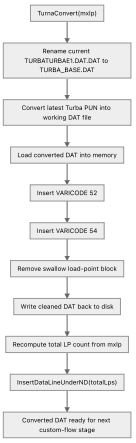
8.7 `cuPunConvertor.TurnaConvert(mxlp)` in-depth flow and purpose

This method is the **file-conversion and DAT re-preparation bridge** used after the custom nozzle optimizer finishes.

In simple terms, it takes the latest Turba-generated `.PUN` result, converts it back into the working `.DAT` form, inserts the custom varicode lines needed for the next phase, removes the swallow LP block, and fixes the load-point count again.

So this method is not doing optimization. It is turning the optimizer output into the **next valid working DAT** for downstream custom checks and final runs.

8.7.1 Easy overall picture

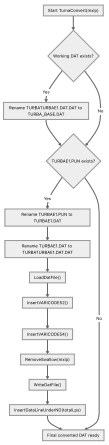


8.7.2 What this method is doing, step by step

1. **Preserve the current DAT as a base copy**
   - `RenameOLDDat()` renames:
     - `TURBATURBAE1.DAT.DAT` -> `TURBA_BASE.DAT`
   - This keeps the previous working DAT as a base/reference before the `.PUN` output is turned into a new DAT.
2. **Convert Turba output into a DAT again**
   - `ConvertPUN()` performs a two-step rename:
     - `TURBAE1.PUN` -> `TURBAE1.DAT`
     - `TURBAE1.DAT` -> `TURBATURBAE1.DAT.DAT`
   - In other words, the latest Turba result becomes the new working DAT file.
3. **Load the converted DAT**
   - `LoadDatFile()` reads the converted `TURBATURBAE1.DAT.DAT` into `turbineDataModel.DAT_DATA`.
   - From this point on, edits happen in memory first.
4. **Insert required custom varicodes**
   - `InsertVARICODE52()` finds the `49.000` varicode line and inserts a new `52.000` line after it.
   - `InsertVARICODE54()` then finds the new `52.000` line and inserts a `54.000 13200.000` style line after it.
   - These insertions prepare the converted DAT for the next custom-flow behavior expected by the codebase.

5. **Remove the swallow load-point block**
    - `RemoveSwallow(mxlp)` removes the LP block starting from:
        - `!LP<mxlp>` if `mxlp > 0`
        - otherwise `!LP11`
    - The method clears four lines starting at that LP marker, then compacts the list by removing empty lines.
    - The purpose is to strip out the swallow LP block that should not remain in the converted DAT for the next stage.
6. **Write the edited DAT back**
    - `WriteDatFile()` writes `turbineDataModel.DAT_DATA` back to disk.
7. **Fix the ND / load-point count**
    - `totalLps = (mxlp >= 1) ? (mxlp – 1) : 0`
    - `InsertDataLineUnderND(totalLps)` updates the ND section so the DAT metadata matches the LP structure left after swallow removal.

8.7.3 Detailed conversion flow



8.7.4 How `RemoveSwallow(mxlp)` decides what to delete

This part is important because `mxlp` directly affects the cleanup.

- If `mxlp > 0`, the method looks for:
    - `!LP<mxlp>`
- Otherwise it defaults to:
    - `!LP11`

Once it finds that LP marker, it removes that LP block by blanking four lines and then filtering empty lines out of the DAT list.

So the meaning is:

- use the actual last custom LP when known,
- otherwise assume the swallow block starts at LP11.

8.7.5 Why varicode insertion happens here

The conversion step is not just a file rename.

The code also adds:

- `VARICODE 52`

- `VARICODE 54`

This suggests the converted DAT must carry extra control/configuration entries before the next Turba/custom validation phase runs.

So `TurnaConvert()` is both:

- a **file conversion** step, and
- a **DAT patching** step.

8.7.6 Why the ND line is fixed again at the end

After swallow removal, the DAT may no longer have the same effective number of active LPs.

That is why the method ends with:

- recomputing `totalLps`
- calling `InsertDataLineUnderND(totalLps)`

Without this, the LP count metadata in the DAT could disagree with the actual LP blocks present in the file.

8.7.7 Easy summary

`TurnaConvert(mxlp)` is the method that:

1. preserves the old DAT,
2. converts the latest Turba `.PUN` output into the new working DAT,
3. injects custom varicode entries,
4. removes the swallow LP block,
5. rewrites the DAT,
6. fixes the LP count metadata.

So in one sentence: this function is the **post-optimizer DAT conversion step that transforms Turba output back into a clean custom-runtime DAT for the next engineering checks and launches**.

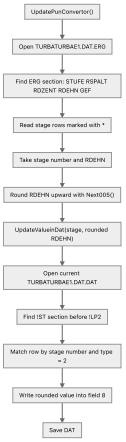
8.8 `cuPunConvertor.UpdatePunConvertor()` in-depth flow and purpose

This method is the **ERG-to-DAT stage update step** that runs after `TurnaConvert(mxlp)`.

In simple terms, it reads stage-wise deformation data from the Turba `.ERG` file, rounds those values upward to the next `0.05`, and writes them back into the current working DAT stage table.

So this is not a general DAT rebuild. It is a **targeted stage-parameter synchronization step** that transfers important stage results from ERG into DAT.

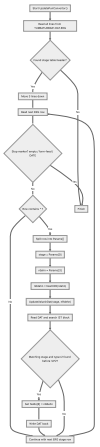
8.8.1 Easy overall picture



8.8.2 What this method is doing, step by step

1. **Open the ERG file**
   - The method reads:
     - `C:\testDir\TURBATURBAE1.DAT.ERG`
2. **Find the stage-result table**
   - It looks for the header:
     - `STUFE  RSPALT  RDZENT  RDEHN  GEF`
   - That is the ERG section containing stage-wise data.
3. **Skip down into the actual rows**
   - After finding the header, the method moves down a few lines.
   - Then it loops row by row until it reaches an empty line, form-feed marker, or `DATE`.
4. **Process only marked stage rows**
   - Only lines containing `*` are processed.
   - For each such row, it extracts:
     - stage number from `Params[0]`
     - `RDEHN` from `Params[3]`
5. **Round `RDEHN` upward**
   - `Next005()` converts the ERG value to the **next higher multiple of `0.05`**.
   - If the value is already exactly on a `0.05` step, it still adds one more `0.05`.
6. **Write the updated value into the DAT**
   - `UpdateValueinDat(find, replace)` opens:
     - `C:\testDir\TURBATURBAE1.DAT.DAT`
   - It searches the stage table under `!ST`.
   - For each stage row before `!LP2`, it splits the row by `|`.
   - It updates the row where:
     - field `0` = matching stage number
     - field `1` = `2`
   - Then it replaces:
     - `fields[8] = replace.ToString("0.00")`
7. **Save the DAT**
   - After the matching stage row is modified, the DAT file is written back to disk.

8.8.3 Detailed update flow



8.8.4 What `Next005()` is doing

This helper is small but very important.

Its rule is:

- take a numeric value,
- move it to the **next higher** `0.05` **step**,
- even if it is already exactly on a step, move one more step higher.

Examples:

- `1.02` -> `1.05`
- `1.05` -> `1.10`
- `1.11` -> `1.15`

So the method is intentionally **conservative upward rounding**, not simple normal rounding.

8.8.5 Why the DAT update is limited to the `!ST` block before `!LP2`

`UpdateValueinDat()` only edits rows:

- inside the stage section starting at `!ST`
- before the next `!LP2`
- where the second field equals `2`

This means the method is carefully targeting one particular stage-data region in the DAT, instead of globally replacing numbers everywhere.

That is important because the same stage number might appear in other parts of the file, but this method only wants the **main stage table entries** relevant for this conversion step.

8.8.6 Why this step matters in the custom flow

After Turba has run, the ERG file contains updated stage information that the DAT does not yet fully reflect.

`UpdatePunConvertor()` closes that gap by:

- reading stage-wise ERG output,
- converting the deformation value into the format/range expected by DAT,
- syncing it back into the DAT stage table.

So this is effectively a **feedback step from ERG to DAT**.

8.8.7 Easy summary

`UpdatePunConvertor()` is the method that:

1. reads stage `RDEHN` values from the ERG,
2. rounds them upward to the next `0.05`,
3. finds the matching stage rows in the DAT,
4. writes the rounded values back into the DAT,
5. saves the file again.

So in one sentence: this function is the **stage-data feedback updater that pushes ERG deformation results back into the working DAT before the next custom-flow steps continue**.

8.9 Custom power closure: `checkFinalTurbine` + `CustomPowerMatch.CheckPower()` + `CorrectLP5Bending()` (easy spec)

The custom path closes by running **custom power match + final checks** (`checkFinalTurbine`). Inside that closure, the code also uses `CustomPowerMatch.CheckPower(maxLp)` (from `src/core/Checks/Cu_ERG_PowerMatch.cs`). A key helper used *inside that power-match loop* is `CorrectLP5Bending()` .

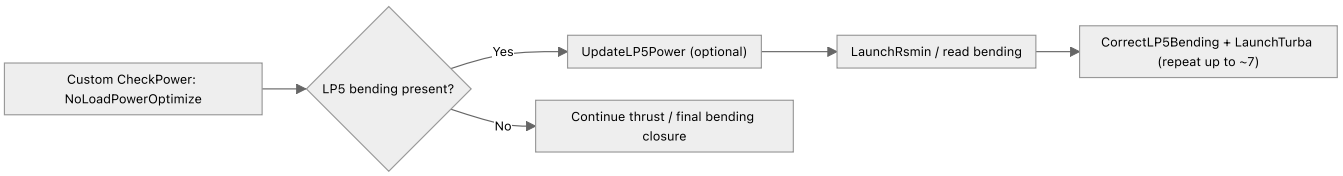
What `CorrectLP5Bending()` does (same concept as executed, custom formatting):

- It reads the LP5 stage-status table from `TURBATURBAE1.DAT.ERG` under the `#UST 5` block.
- If any stage row has a bending flag token `B` or `F` near the end of the row, it patches the corresponding stage row in the **Turba DAT blade table** (`TURBATURBAE1.DAT.DAT`).
- The patch is applied by calling:
  - `CorrectDatFileB(stage, gi)` for `B`
  - `CorrectDatFileF(stage, gi)` for `F`

Why it exists:

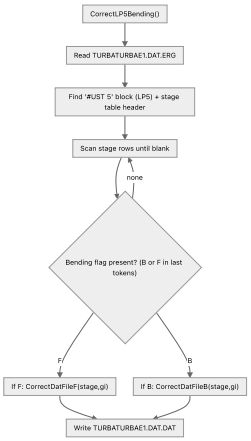
- In custom flow, LP5 is again the “hard” point used to validate bending-sensitive behavior.
- Instead of abandoning the run immediately on an LP5 bending flag, the code attempts a **deterministic DAT edit** to move the design toward a pass on the next Turba rerun.

Where it sits in the custom `CheckPower` loop (very high-level):



Custom vs executed differences you’ll see in code:

- The custom DAT blade table uses a different parsing format in places (rows split by `|` in the custom patchers), but the intent is the same:
  - identify the stage by `(stage, gi)` keys coming from the ERG row,
  - adjust the blade-table row so bending status improves,
  - write the DAT back to disk.



9) Flow-wise content: Additional load points path

> **Overview chart:** Additional load points flowchart in the flowcharts gallery.

This path appears when the main flow has **more than one customer load point** to merge into Kreisl/Turba work (exact gate depends on caller: e.g. executed path checks `CustomerLoadPoints.Count > 2` in places; the idea is the same: **extra LPs beyond the base case**).

Implementation lives mainly in `AdditionalLoadPoints.cs` (`CustomLoadPointHandler`), especially **`cxLP_mainKreisl(customerLPList)`**.

9.0.1 Legend (terms used in charts)

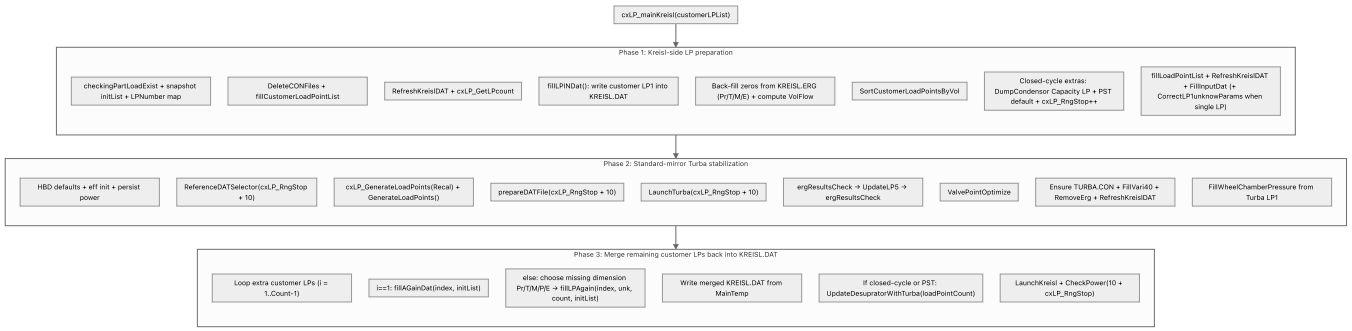
- **LP indexing**
  - **LP1** in this chapter refers to the *first customer load point row the handler uses* (in code many accesses use `CustomerLoadPoints[1]` as the base row).
  - "extra LPs" means subsequent customer points (`i = 2..` in the spreadsheet sense), merged via `fillAGainDat` / `fillLPAgain`.
- **Unknown-dimension codes**
  - **`Pr`** : `SteamPressure` is unknown / zero
  - **`T`** : `SteamTemp` is unknown / zero
  - **`M`** : `SteamMass` is unknown / zero
  - **`P`** : `PowerGeneration` is unknown / zero
  - **`E`** : `ExhaustPressure` is unknown / zero
- **Key artifacts**
  - **`KREISL.DAT`** (written repeatedly) and **`KREISL.ERG`** (read to back-fill)
  - **`TURBATURBAE1.DAT.ERG`** (read during desuperheater update)
  - **`MainTemp`** : an in-memory accumulator that often does `MainTemp += File.ReadAllText("C:\\testDir\\KREISL.DAT")` after each LP write.

9.1 High-level flow (**`cxLP_mainKreisl`**)

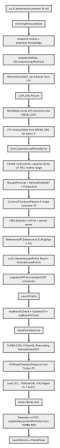
If you open `src/AdditionalLoadPoints.cs`, it can feel confusing because **two different pipelines are stitched together**:

- **Phase 1 (Kreisl-side LP preparation)**: take customer LPs, write LP1 into `KREISL.DAT`, run Kreisl as needed, and back-fill missing values from `KREISL.ERG`, then sort LPs by volumetric flow.
- **Phase 2 (Standard-mirror Turba block)**: run the normal **Reference DAT → internal LPs → Turba → ERG → UpdateLP5 → ERG → valve** sequence to stabilize the design at the chosen internal points.
- **Phase 3 (Per-customer-LP merge loop)**: for each extra customer LP, write/update the Kreisl DAT block using `fillAGainDat` / `fillLPAgain` until all customer LPs are merged, then do final desuperheater sync (if closed/PST) and close with Kreisl + `CheckPower`.

9.1.1 Main flow (phased and readable)

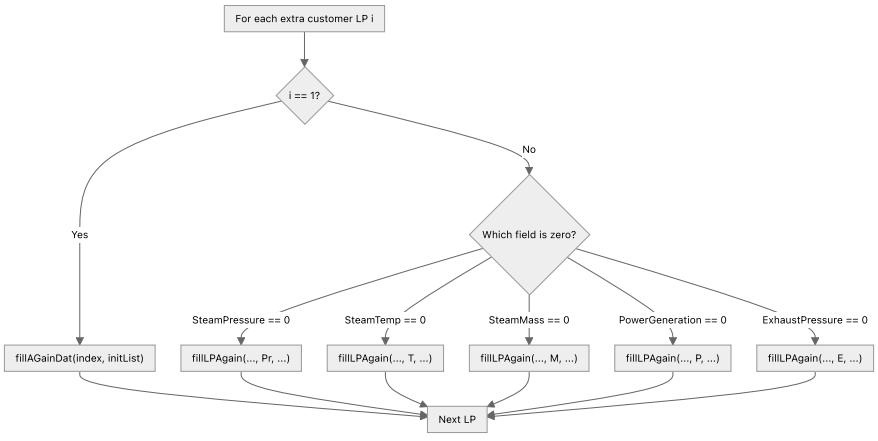


9.1.2 Full chain chart (original “single line” view)



9.2 Per-LP merge after Turba (unknown dimension)

After the Turba stabilization block, the handler walks **each extra customer LP** (`i = 1 .. Count-1`). For `i == 1` it rebuilds the first appended block via `fillAGainDat` . For later indices it picks the unknown and calls **`fillLPAgain(index, dimension, lpRow, initList)`** with `Pr` , `T` , `M` , `P` , or `E` .



Open-cycle mass-flow tie-up (when neither deaerator nor PST): if mass is unknown but exhaust mass is known, code uses **`SteamMass = 0.055 + ExhaustMassFlow`** ; if mass is known but exhaust mass is not, **`ExhaustMassFlow = SteamMass – 0.055`** .

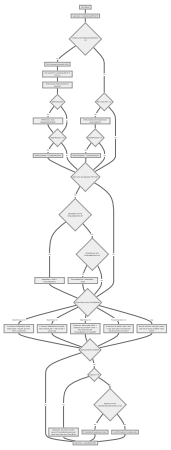
9.3 LP1 into Kreisl (**`fillLPINDat`**)

Before the ERG fill loop, **`fillLPINDat()`** writes customer LP1 boundary conditions into **`KREISL.DAT`** via `KreislDATHandler` (pressure, temperature, mass flow, power, exhaust pressure, closed-cycle makeup/condensate/PST/PRV branches, dump condenser capacity paths). It accumulates working text in **`MainTemp`** (often by appending the current `KREISL.DAT` file after edits).

9.3.1 `fillLPINDat()` mini spec (inputs/outputs)

- **Reads**
  - `AdditionalLoadPoint.GetInstance().CustomerLoadPoints[1]` fields (pressure/temp/mass/power/exhaust pressure/exhaust mass/PST/closed-cycle fields)
  - `turbineDataModel` flags: `DeaeratorOutletTemp`, `PST`, `DumpCondensor`, `IsPRVTemplate`, `ExhaustPressure`
- **Writes**
  - `StartKreisl.filePath` (the Kreisl DAT) via `KreislDATHandler.*`
  - appends the final file text into `MainTemp`
- **Key behavior**
  - If **closed-cycle** (`DeaeratorOutletTemp > 0`) it writes makeup/condensate/process steam temp (PST) and optionally desuperheater pressure.
  - If **PST-only** (`PST > 0`) it writes process steam temp and optional desuperheater pressure.
  - If **open-cycle** (no deaerator and PST=0) it applies the `0.055` mass tie-up between inlet and exhaust mass flows.

9.3.2 Detailed branch chart for `fillLPINDat()`



9.4 ERG back-fill for all customer LPs

After LP1 is in the DAT, a loop over **`CustomerLoadPoints[1..]`** fills any zero from **`KREISL.ERG`** using `KreislERGHandlerService` (`ExtractPressure`, `ExtractTemperature`, `ExtractMassFlow`, `ExtractBackPressure`, `ExtractVolFlowForLoadPoint`). Then **`SortCustomerLoadPointsByVol()`** reorders LPs by volumetric flow.

9.5 Standard-path DAT rebuild and checks

The middle block mirrors the **standard** pipeline at scaled LP count:

- `ReferenceDATSelector((int)cxLP_RngStop + 10)`
- `cxLP_GenerateLoadPoints("Recal")` then `GenerateLoadPoints()`
- `prepareDATFile` -> `DATFileProcessor.PrepareDATFile`
- `LaunchTurba`
- `ergResultsCheck` -> `UpdateLP5` -> `ergResultsCheck` again (LP5 flag on `ERGVerification`)
- `ValvePointOptimize`

9.6 Desuperheater update from Turba ERG

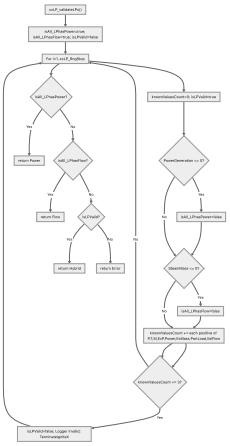
If **`DeaeratorOutletTemp > 0`** or **`PST > 0`**, **`UpdateDesupratorWithTurba(...)`** reads **`TURBATURBAE1.DAT.ERG`** pressure/temperature/enthalpy/mass blocks and updates Kreisl desuperheater sections per LP (closed PRV with/without dump condenser, or open-cycle desuperheater helpers), then **`LaunchKreisl`** and **`CheckPower`**.

9.7 LP validation helper (`cxLP_validateLPs`)

Used to classify customer input before heavy work: counts known fields per LP (must be more than three knowns per LP in the current rule), and returns `Power`, `Flow`, `Hybrid`, or `Error` depending on whether all LPs have power, all have mass flow, or a mix.

9.7.1 Detailed decision chart for `cxLP_validateLPs()`

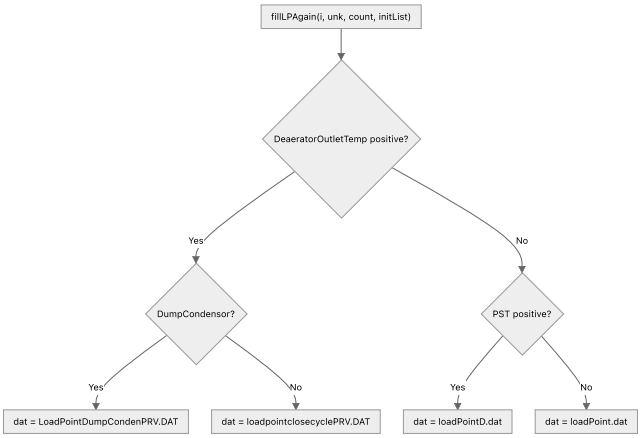
In code, "known" increments for: `SteamPressure`, `SteamTemp`, `SteamMass`, `ExhaustPressure`, `PowerGeneration`, `ExhaustMassFlow`, `PartLoad`, `VolFlow` (each positive adds 1). If `knownValuesCount <= 3` for any LP, it logs and terminates.



9.8 Inner flow: `fillLPAgain(i, unk, count, initList)`

Used when **LP index `i` is not the first** extra LP and one dimension is still unknown (`unk` is `Pr`, `T`, `M`, `P`, or `E`). It picks a **row template file**, stamps the Kreisl LP row id, then writes boundary guesses into `KREISL.DAT` via `KreislDATHandler`, and appends the updated file into `MainTemp`.

9.8.1 Choose LP row template (`dat` path)

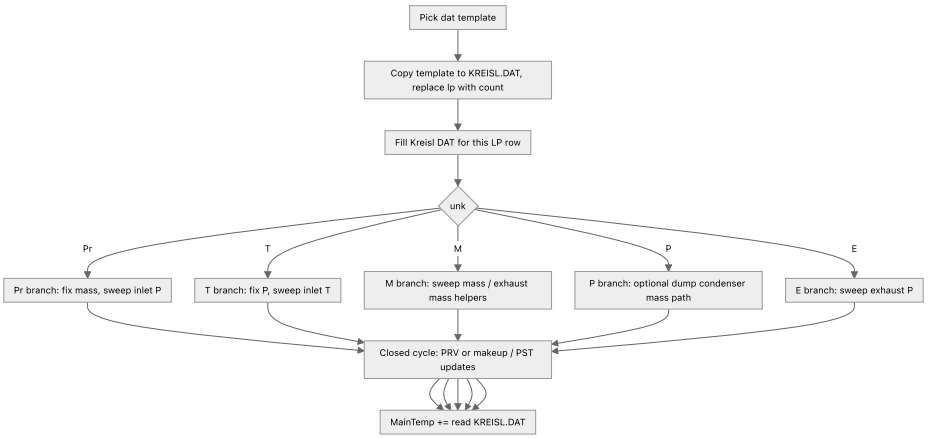


9.8.2 Unknown-dimension branches (same pattern for each `unk`)

For every `unk`, the code does:

1. copy the chosen template to `C:\testDir\KREISL.DAT`, clear it, read template text, replace `lp` -> `count`, append to `KREISL.DAT`
2. fill the **known** inlet fields on `StartKreisl.filePath`
3. drive the **unknown** field with a Kreisl sweep pattern (for example `Pr`: mass fixed, inlet pressure stepped `0.000` then `42.981`; `T`: temperature stepped `0.000` then `440`; `E`: exhaust pressure stepped `0.000` then `4.59`)
4. optional **dump condenser** branch: capacity vs `checkIfDumpcondensorON` vs `TurnOffCondensor` lines

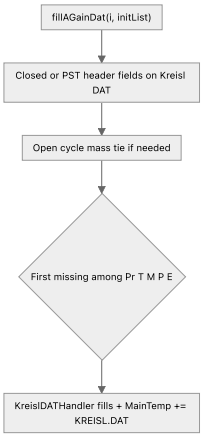
5. closed-cycle tail: PRV to WPRV conversion, makeup/condensate/PST lines, `FillPressureDesh` when pressure is known and `unk` is not `Pr`
6. **`MainTemp +=`** **entire** **`KREISL.DAT`** after edits



9.9 Inner flow: `fillAGainDat(i, initList)`

Same physical idea as **`fillLPINDat`**, but for **customer index** **`i`** **inside** **`initList`** when rebuilding the **first** extra LP block (`i == 1` path in `cxLP_mainKreisl`). It:

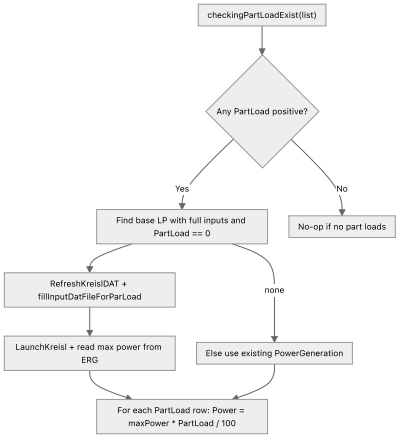
1. applies **closed-cycle** makeup / PST / PRV logic when `initList[i].DeaeratorOutletTemp > 0`, or **PST-only** desuperheater pressure when `PST > 0`
2. applies **open-cycle** mass tie (`0.055` rule) when neither deaerator nor PST
3. runs the same **Pr / T / M / P / E** ladder as `fillLPAgain`, writing Kreisl and appending **`MainTemp`** for whichever branch matches the missing field



9.10 Inner flow: `checkingPartLoadExist(customerLoadPoints)`

If **any** customer row has **`PartLoad > 0`**, the method:

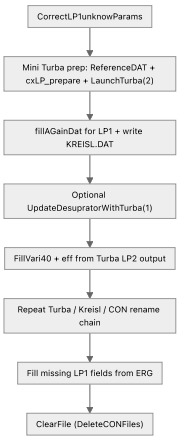
1. scans for a **fully specified base LP** (`SteamPressure`, `SteamTemp`, `SteamMass`, `ExhaustPressure` all positive and **`PartLoad == 0`**)
2. for that row: `RefreshKreislDAT`, `fillInputDatFileForParLoad`, rename Turba CON, **`LaunchKreisL`**, read **`ExtractPowerFromERG`** as candidate max power
3. for rows without full thermo, it keeps **`PowerGeneration`** from the sheet as the max candidate
4. for every row with **`PartLoad > 0`**, sets
   `PowerGeneration = (maxPower * PartLoad) / 100`



9.11 Inner flow: `CorrectLP1unknowParams()`

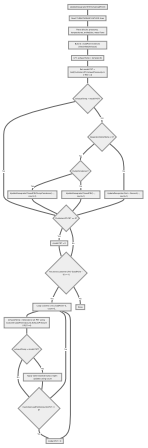
Runs only when **`customerLPList.Count == 1`** inside `cxLP_mainKreisl`. It is a **mini calibration loop** for LP1 before the big multi-LP pipeline:

1. `HBDsetDefaultCustomerParamas`, `cxLP_GenerateLoadPoints`, `ReferenceDATSelector(1)`, `cxLP_prepareDATFile(1)`, `LaunchTurba(2)`
2. optional wheel chamber pressure write-back
3. **`fillAGainDat`** for LP1, then **`File.WriteAllText(KREISL.DAT, MainTemp)`**
4. optional **`UpdateDesupratorWithTurba(1)`** when closed cycle or PST
5. `FillVari40`, copy Turba efficiency into Kreisl eff fields, then **several Turba + Kreisl launch pairs** with CON rename between them
6. fills any remaining LP1 unknown from **`KREISL.ERG`**, then **`ClearFile`** (deletes CON side artifacts)



9.12 Inner flow: `UpdateDesupratorWithTurba(loadPoint)` (summary)

Parses **`TURBATURBAE1.DAT.ERG`** blocks (`DRUECKE`, `TEMPERATUREN`, `ENTHALPIEN`, `DAMPFMENGEN`) into per-LP matrices, compares **exhaust temperature vs PST** per LP, and calls the matching **`KreislDATHandler.UpdateDesuprator...`** helpers (open cycle, closed PRV, dump condenser variants). Resets **`turbineDataModel.PST`** when customer PST is zero on LP1 or each extra LP.



10) Complete flow map (all paths)

`Main Entry` -> `Standard` or `Executed` or `Custom`

- `Standard` -> single baseline path -> ERG checks -> valve optimization -> power match.
- `Executed` -> `BCD1120 / BCD1190 / Throttle` criteria sub-flow -> fallback chain -> power match.
- `Custom` -> custom DAT + PSO + custom ERG criteria -> valve + final custom checks.
- `Standard/Executed/Custom` + additional LP count -> `Additional Load Points LP-wise loop`.

At-a-glance Mermaid diagrams for each branch: Flowcharts gallery.

11) Code documentation — execution starting points

This section maps **where execution actually starts in code** (types, files, and typical callers) so you can align the flowcharts with the repository. The UI host (Ignite X / MAUI) generally wires button or pipeline actions to these methods; search the solution for the method name to find every call site.

11.1 Primary entry methods (by path)

Path	Type / method	File
Standard (UI-style pipeline)	`StartExec.Main4`	`src/Program.cs`
Standard (Kreisl-first orchestration)	`StartKreisl.MainKreisL`	`src/kreisl.cs`
Prefeasibility → branch	`DatFileSelector.ReferenceDATSelector`	`src/core/HMBD/Ref_DAT_select`

Path	Type / method	File
Executed	`MainExecutedClass.MainExecuted(criteria, maxLp)`	`src/Main_Executed.cs`
Executed (shortcuts)	`GotoBCD1120`, `GotoBCD1190`	`src/Main_Executed.cs`
Custom	`CustomExecutedClass.Main_CustomFlowPathTest(mxlp)`	`src/Main_Custom.cs`
Additional load points	`CustomLoadPointHandler.cxLP_mainKreisl(customerLPList)`	`src/AdditionalLoadPoints.cs`

11.2 Dispatch diagram (conceptual — who calls whom)

Solid arrows are representative; actual wiring depends on the active page and LP count — use IDE **Find references** on each method name for exact callers.

11.3 Supporting singletons / config loaded at startup

- **`StartExec.InitConfig`** (`src/Program.cs`): fills nozzle, power-efficiency, pre-feasibility, load-point and Turba output models from Excel/backend data before `FillInputValues` updates Kreisl DAT fields.
- **DI registration** (`StartExec.CreateHostBuilder`): `IThermodynamicLibrary`, `ILogger`, `IERGHandlerService` shared by Kreisl and standard paths.

11.4 How to extend this documentation

Later **method-by-method** chapters can cite: **caller → callee → condition → flowchart subsection**. The gallery links **Standard → Sections 5–6**, **Executed → Section 7**, **Custom → Section 8**, **Additional LP → Section 9**.

11.5 End-to-end code walkthrough (by path)

This section is the "**how the code actually runs**" narrative: **entry point → major calls → handoffs**. It is intentionally shorter than Sections 5–9 (which are diagram-heavy and method-deep).

11.5.1 Standard flow (two entry points)

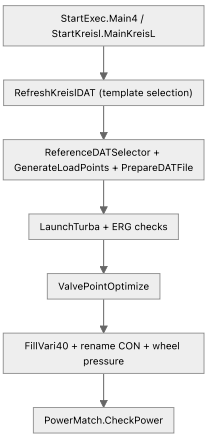
There are two common "standard" entry shapes:

- **UI-style pipeline**: `StartExec.Main4(args)` in `src/Program.cs`
- **Kreisl-first orchestration**: `StartKreisl.MainKreisL(args)` in `src/kreisl.cs`

Both do the same *big idea*: **select template → run Kreisl/Turba → validate ERG → optimize valve → close power**. Details are in **Section 5 (flowchart)** and **Section 6 (theory)**.

Call-tree sketch:

- `StartExec.Main4`
  - host + config
  - `StartKreisl.DeleteCONFiles`
  - `KreislDATHandler.RefreshKreislDAT` (template family selection)
  - `InitConfig`
  - `HBDPowerCalculator` defaults + init
  - `DatFileSelector.ReferenceDATSelector`
  - `LoadPointGen.GenerateLoadPoints`
  - `DATFileProcessor.PrepareDATFile`
  - `TurbaConfig.LaunchTurba`
  - `ERGVerification.ErgResultsCheck`
  - `ValvePointOptimizer.ValvePointOptimize`
  - `PowerMatch.CheckPower`



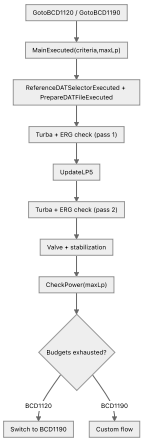
11.5.2 Executed flow (criteria controller)

Executed flow is controlled by `MainExecutedClass` (`src/Main_Executed.cs`):

- Entry methods:
  - `GotoBCD1120(maxLp)` → calls `MainExecuted("BCD1120", maxLp)`
  - `GotoBCD1190(maxLp)` → calls `MainExecuted("BCD1190", maxLp)`
- Core method:
  - `MainExecuted(criteria, maxLp)` where `criteria ∈ { BCD1120, BCD1190, Throttle }`

High-level shape (details are in **Section 7**):

- retry/fallback gates (budgets)
- nearest executed project selection + executed reference DAT copy
- executed LP generation + executed DAT rebuild
- Turba run + ERG check pass 1
- `UpdateLP5` + ERG check pass 2
- valve optimization + stabilization
- `CheckPower(maxLp)`
- fallback handoffs:
  - `BCD1120` budget exhausted → rerun as `BCD1190`
  - `BCD1190` budget exhausted → `CustomExecutedClass.Main_CustomFlowPathTest(maxLp)`

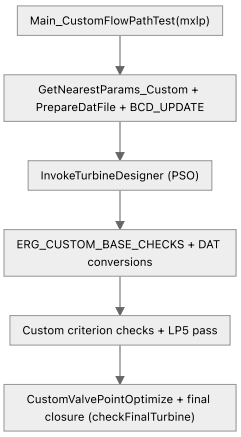


11.5.3 Custom flow (PSO + custom checks)

Custom flow entry is `CustomExecutedClass.Main_CustomFlowPathTest(mxlp)` in `src/Main_Custom.cs`.

High-level shape (details are in **Section 8**):

- cleanup + `RefreshKreislDAT`
- generate custom LPs
- pre-feasibility decisions
- nearest custom params + custom reference DAT copy
- custom DAT rebuild (`PrepareDatFile`)
- BCD rewrite (`BCD_UPDATE`)
- PSO optimizer loop (propose knobs → Turba → penalty checks → update particles)
- base custom checks + DAT conversions (`TurnaConvert` , `UpdatePunConvertor`)
- custom ERG criterion checks (1120 vs 1190), LP5 update + re-check
- custom valve optimization + final closure (`checkFinalTurbine`)



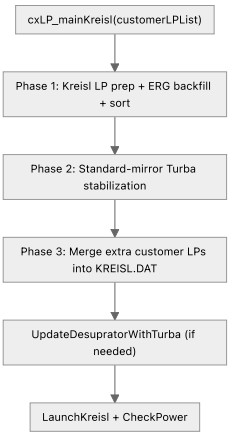
11.5.4 Additional load points flow (customer LP merge path)

Additional-load-points entry is `CustomLoadPointHandler.cxLP_mainKreisl(customerLPList)` in `src/AdditionalLoadPoints.cs`.

High-level shape (details are in **Section 9**):

- **Phase 1**: write customer LP1 into `KREISL.DAT` , back-fill zeros from `KREISL.ERG` , sort by volumetric flow
- **Phase 2**: run a **standard-mirror Turba stabilization** block (`ReferenceDATSelector → Turba → ERG → LP5 → valve`)
- **Phase 3**: loop each extra customer LP and append/repair the Kreisl DAT text using `fillAGainDat` / `fillLPAgain(Pr/T/M/P/E)`

- finalize: optional desuperheater update from Turba ERG, `LaunchKreisl`, then `CheckPower(...)`



11.6 Code documentation (deeper): function-by-function walkthroughs

This is the "**read the code in the same order it executes**" documentation. For each path, you get:

- **function name** (exact method as in code)
- **what it does**
- **what it reads/writes (important files)**
- **what it calls next**

11.6.1 Standard path (UI-style) — `StartExec.Main4(args)` in `src/Program.cs`

Execution order (simplified but accurate to code):

1. `StartExec.CreateHostBuilder(args).Build()`
   - **does**: sets up DI container
   - **provides**: `IThermodynamicLibrary`, `ILogger`, `IERGHandlerService`
2. **Read config** (`src/core/Config/appsettings.json`) and set `excelPath`
3. `StartKreisl.FillGlobalHost()` *(if needed)*
   - **does**: ensures Kreisl-side host exists (Kreisl/Turba services available)
4. `StartKreisl.DeleteCONFiles()`
   - **does**: deletes old `.CON/.ERG` artifacts from `C:\testDir` and `C:\testDir\Turman250`
5. `new KreislDATHandler().RefreshKreislDAT()`
   - **does**: picks the correct Kreisl template family (open/PST vs closed/PRV/dump) and copies/updates the working Kreisl DAT
   - **details**: see **Section 6.2** (template selection)
6. `StartExec.InitConfig()`
   - **does**: loads the Excel-backed models (`NozzleTurbaDataModel`, `PowerEfficiencyModel`, `PreFeasibilityDataModel`, `LoadPointDataModel`, `TurbaOutputModel`)
   - **then calls**: `StartExec.FillInputValues()` which writes inlet/exhaust/mass into the working Kreisl DAT via `KreislDATHandler.Fill*`
7. `HBDPowerCalculator.HBDSetDefaultCustomerParams()`
   - **does**: seeds HMBD defaults for this run (customer parameters)
8. `DatFileSelector.ReferenceDATSelector()`
   - **does**: chooses the reference Turba DAT flowpath based on **pre-feasibility** decisions and copies it into working area
   - **details**: see **Section 7.6** (executed selector) and **Section 6.0/6.4** (standard spine)
9. `LoadPointGen.GenerateLoadPoints()`
   - **does**: generates internal load points used by Turba DAT writing
10. `DATFileProcessor.PrepareDATFile()`

- **does**: writes the generated load points + init parameters into `TURBATURBAE1.DAT.DAT`

11. **`TurbaConfig.LaunchTurba()`**

- **does**: runs Turba.exe with the current DAT and produces `TURBATURBAE1.DAT.ERG` and `TURBATURBAE1.DAT.CON`

12. **`ERGVerification.ErgResultsCheck()`**

- **does**: validates the Turba ERG outputs against engineering constraints (criterion checks, limits, etc.)

13. **`ValvePointOptimizer.ValvePointOptimize()`**

- **does**: iteratively adjusts valve/nozzle group conditions to reduce deviation and improve convergence

14. **`PowerMatch.CheckPower()`**

- **does**: final closure; includes no-load checks, bending/thrust checks, and repair loops (LP5 logic)
- **deep dive**: see **Section 7.10.1** (`CorrectLP5Bending` and DAT patchers)

11.6.1.1 Deeper: what each Standard step calls (mini call trees)

Below, each block shows the **sub-functions / key helpers** called by that step in the current implementation.

A) **`StartExec.CreateHostBuilder(args).Build()`**

- `StartExec.CreateHostBuilder(args)`
  - `.ConfigureServices(...)`
    - `services.AddSingleton<IThermodynamicLibrary, ThermodynamicService>()`
    - `services.AddSingleton<ILogger, Logger>()`
    - `services.AddSingleton<IERGHandlerService, KreislERGHandlerService>()`

B) **`StartKreisl.FillGlobalHost()`** **(if `StartKreisl.GlobalHost == null`)**

- `StartKreisl.CreateHostBuilder(null).Build()`
- reads `src/core/Config/appsettings.json` and sets `StartKreisl.excelPath`
- assigns `StartExec.GlobalHost = StartKreisl.GlobalHost`

C) **`StartKreisl.DeleteCONFiles()`**

- deletes `*.CON` and `*.ERG` under:
  - `C:\testDir`
  - `C:\testDir\Turman250` *(if directory exists)*

D) **`KreislDATHandler.RefreshKreislDAT()`**

At a high level this method does:

- reads cycle mode flags from `TurbineDataModel`:
  - `DeaeratorOutletTemp`, `DumpCondensor`, `PST`, `ExhaustPressure`
- may read `C:\testDir\KREISL.ERG` to extract `exhaustTemp` (`ExtractTempForDesuparator(...)`)
- copies one template from `AppContext.BaseDirectory` into `C:\testDir\KREISL.DAT`
- may call one of these converters:
  - `UpdateTemplatePRVToWPRVInDumpCondensor(StartKreisl.filePath)`
  - `UpdateTemplatePRVToWPRV(StartKreisl.filePath)`
- sets `turbineDataModel.IsPRVTemplate` accordingly

> This is exactly why Section 6.2 uses diagram labels like `T1/T2/T7` — the implementation is a nested decision tree.

E) **`StartExec.InitConfig()`**

- `NozzleTurbaDataModel.getInstance().fillNozzleTurbaDataModel()`
- `PowerEfficiencyModel.getInstance().fillPowerEfficiencyDataModel()`
- `PreFeasibilityDataModel.getInstance().fillPreFeasibilityData()`
- `LoadPointDataModel.getInstance().fillLoadPoints()`
- `TurbaOutputModel.getInstance().fillTurbaOutputDataList()`
- sets:
  - `turbineDataModel.GeneratorEfficiency = getGeneratorEff()`
  - `turbineDataModel.LeakagePressure = 1.015`
- calls **`StartExec.FillInputValues()`**
  - `KreislDATHandler.FillMassFlow(StartKreisl.filePath, ...)`
  - `KreislDATHandler.FillInletPressure(StartKreisl.filePath, ...)`
  - `KreislDATHandler.FillExhaustPressure(StartKreisl.filePath, ...)`
  - `KreislDATHandler.FillInletTemperature(StartKreisl.filePath, ...)`

F) `HBDPowerCalculator.HBDSetDefaultCustomerParams()`

- seeds default customer/HMBD parameters (implementation lives in `HMBDInformation` / HMBD configuration classes)
- prepares the state used by efficiency/power persistence steps that follow

G) `DatFileSelector.ReferenceDATSelector()`

- `DatFileSelector.getFlowPath(maxLp)`
  - `PreFeasibilityDataModel.fillPrefeasibilityDecisionChecks()`
  - `SelectStandard("Straight", maxLp)` → sets `preFeasibilityDataModel.Variant`
  - `CopyRefDATFile(path)` to copy the chosen `TURBATURBAE1.DAT.DAT` template into working location
  - if standard not feasible:
    - `MainExecutedClass.GotoBCD1190(maxLp)` OR `TurbineDesignPage.cts.Cancel()`

H) `LoadPointGen.GenerateLoadPoints()`

- `KreislIntegration.RenameTurbaCON()` (normalize CON name before using it)
- fills `LoadPointDataModel.LoadPoints[1..]` from `TurbineDataModel` (pressure/temp/mass/backpressure/rpm/flags)
- for MCR-style points, calls:
  - `updateMainTemplate(...)`
  - `KreislIntegration.LaunchKreisL()`
  - `KreislERGHandlerService.ExtractMassFlowFromERGLP9(...)`
  - then continues assembling remaining load points

I) `DATFileProcessor.PrepareDATFile()`

- `LoadLP1FromDAT()`
- `DeleteRowAfterFirstLoadPoint()`
- `InsertDataLineUnderFirstLPFixed()`
- `DeleteLoadPoints()`
- `InsertLoadPointsWithExactFormattingUsingMid(mxLPs)`
- `InsertDataLineUnderND(totalLps)`
- `DatFileInitParamsExceptLP()` (powertrain/nozzles/vari writes)
  - `thermodynamicService.GetInletVelocity(...)`
  - `thermodynamicService.getVolumetricFlow()`
  - `updateGeneratorSpecs(...)` + `HBDPowerCalculator.HBDUpdateEffGenerator(...)`
  - `updateGearboxSpecs(...)`, `updateTurbineSpecs(...)`, `updateVari27(...)`
  - `updateNozzleSpecs(nozzleCount, nozzleFront)`

J) `TurbaConfig.LaunchTurba()`

- if `StartKreisl.kreislKey` and `C:\testDir\KREISL.CON` exists:
  - moves it to `C:\testDir\KREISLTURBAE1.DAT.CON`
- `RunBatchFile(AppContext.BaseDirectory\\auto.bat)` (starts Turba)
- waits for `C:\testDir\TURBA_FLAG.bin`
- on completion:
  - `LoadERGFile(mxLPs)` → `ERGFileReader.LoadERGFile(mxLPs)` (populates `TurbaOutputModel`)
- also scans `C:\testDir\TURBATURBAE1.DAT.LOG` for "Error in input file" and cancels if found

K) `ERGVerification.ErgResultsCheck()`

Runs a gate chain (stops early on first fail):

- `ErgCheckExhaustVolumetricFlow(maxlPS)`
- `ErgCheck_NozzlesSection(maxlPS)`
  - calls `NozzleOptimizer.RuleEngineAlgorithmForNozzles(maxlPS)`
- `ErgCheckDetaTGBCWheelChamberPTBending(maxlPS)`
- `ErgCheckThrustValue(maxlPS)`
  - calls `TurbaConfig.LaunchRsmin()` then checks thrust per LP
- `ErgCheck_LoadPoints(maxlPS)`
  - if stage pressure fails:
    - uses `LoadPointGen.LoadPointGenerator_IncMassFlow(...)` / `LoadPointGenerator_ReduceBP(...)`
    - `DATFileProcessor.PrepareDATFile_OnlyLPUpdate(maxLps)`
    - `TurbaConfig.LaunchTurba(maxLps)` then re-enters `ErgResultsCheck`

Where the LP5 "regen + re-check" happens (Standard):

- In the standard pipelines, this gate chain is run in **two passes**:
  - **Pass 1**: run Turba → run `ERGVerification.ErgResultsCheck()` on the initial load-point set.
  - **Update LP5**: `UpdateLP5()` rewrites LP5 (stress/bending/thrust-oriented point) from the latest turbine state.
  - **Pass 2**: re-run Turba → re-run `ERGVerification.ErgResultsCheck()` again so the same gates (especially bending/thrust-related checks) validate the **new LP5** as well.

L) `ValvePointOptimizer.ValvePointOptimize(maxLPs)`

- reads `TurbaOutputModel` deviation and nozzle-group valve status
- calls one of:
  - `AdjustNozzlePair(...)` → `NozzleOptimizer.UpdateNozzleSpecs(...)` → `TurbaConfig.LaunchTurba()` → re-run `ValvePointOptimize`
  - `AdjustValvePointMassFlow(...)` → `PrepareDATFile_OnlyLPUpdate(...)` → `TurbaConfig.LaunchTurba()` loop until convergence

M) `PowerMatch.CheckPower(maxLP)`

- updates turbine efficiency back into Kreisl/HBD context (`HBDUpdateEffKriesl` / `HBDupdateEff`)
- checks base power delta vs HBD
- runs no-load / bending / thrust closure loops
- includes LP5 bending repair path:
  - `UpdateLP5Power(...)` *(when bending exists)*
  - `TurbaConfig.LaunchRsmin()`
  - `CorrectLP5Bending()` loop (DAT patchers)

This closure logic is detailed in **Section 7.10.1** and **Section 8.9** (custom uses the same bending repair concept).

11.6.2 Standard path (Kreisl-first) — `StartKreisl.MainKreisL(args)` in `src/kreisl.cs`

This is the same “standard idea” but explicitly **runs Kreisl early and resyncs**:

- after `RefreshKreislDAT`, it calls:
  - `thermodynamicService.FillClosestTurbineEfficiency()`
  - `hBDPowerCalculator.GetTurbaCON(ClosestProjectID)`
  - `KreislIntegration.LaunchKreisL()`
  - then `RefreshKreislDAT()` **again +** `InitConfig()` **again** (select → run → resync)
- then it enters the same main block:
  - `ReferenceDATSelector → GenerateLoadPoints → PrepareDATFile → LaunchTurba → ERG pass 1 → UpdateLP5 → ERG pass 2 → ValvePointOptimize → FillVari40 → LaunchTurba → rename CON → FillWheelChamberPressure → PowerMatch.CheckPower`

This is why Section 6.3 calls it **select → run → resync**.

11.6.3 Executed path — `MainExecutedClass.MainExecuted(criteria,maxLp)` in `src/Main_Executed.cs`

Typical entry (outside this method) — callers such as `GotoBCD1120` / `GotoBCD1190` invoke `fillDependencies()` first. That rebuilds `MainExecutedClass.GlobalHost`, fills lookup tables (`NozzleTurbaDataModel`, `ExecutedDB`, `PowerEfficiencyModel`, `PreFeasibilityDataModel`, etc.), reads `AdminControl.csv` for `GeneratorEff` and `NoOfExecuted`, resets logs, seeds `ListPower` from `TurbineDataModel` (notably row 0 = current case boundary conditions), then calls `MainExecuted(...)`.

Execution order inside `MainExecuted`:

1. **Call counters / budgets**
   - `mainCallCounters` for `BCD1120` / `BCD1190` — cap = `turbineDataModel.NoOfExecuted` (neighbor count budget)
   - `throttleCounters` for `Throttle` — hard cap `MAX_THROTTLE_CALLS` (currently `2`); when exceeded the method ** `return` **s *(the console prints “custom path” but ** `Main_CustomFlowPathTest` is not invoked from this branch)*.
   - **Handoffs**
     - `BCD1120` budget exhausted → `ResetCleanUpExecutedNearest()` → `MainExecuted("BCD1190", maxLp)` (fresh attempt with broader neighbor pool rules on the BCD1190 path).
     - `BCD1190` budget exhausted → `Main_CustomFlowPathTest(maxLp)` (custom executed flow).
     - Successful counter increment also `ResetNozzleCounter()` and clears `OldNa` / `OldNb` on `TurbineDataModel`.
2. **Executed HMBD defaults**
   - `ExecHMBDConfiguration`: `HBDsetDefaultCustomerParamas_Executed_Kreisl()` if `StartKreisl.kreislKey`, else `HBDsetDefaultCustomerParamas_Executed()`
3. **Nearest executed project selection**
   - `PowerKNN(criteria)` → `MoveYAndSetParams()`
4. **Reference executed DAT**
   - `ReferenceDATSelectorExecuted(criteria)` → `FlowPathSelector.ReferenceDATSelectorExecuted` in `src/core/HMBD/Exec_Ref_DAT_Selector.cs`.
5. **Load and rebuild DAT**
   - `LoadDatFile()` reads working `TURBATURBAE1.DAT.DAT` into `turbineDataModel.DAT_DATA` (needed for RADKAMMER / parameter scrape).
   - `GenerateLoadPoints(maxLp)` then `HBDsetDefaultCustomerParamsExecuted(kreislKey)` — second pass on executed HMBD defaults after LP generation starts.
   - `HBDupdateEfficiency` copies `ListPower[0].Efficiency` into `turbineDataModel.TurbineEfficiency`.
   - `PrepareDATFileExecuted(maxLp)`
6. **Wheel chamber guard**
   - `IsWheelChamberPressureValid()` compares `RADKAMMER` from in-memory `DAT_DATA`, `PreFeasibilityDataModel` inlet/back-pressure against engineering limits; **if false**, logs and **recursively calls** `MainExecuted(criteria, maxLp)` so `MoveYAndSetParams` can advance to another neighbor (`FlowPathSelector.AddOrMoveY` side effects).

7. **Turba + ERG pass 1**
   - `LaunchTurba(maxLp)` — `TurbaAutomation.LaunchTurba` in `src/core/Turba/Exec_TurbaConfig.cs` (moves `KREISL.CON` → `KREISLTURBAE1.DAT.CON` when present, runs batch, loads ERG into `TurbaOutputModel`).
   - `ErgResultsCheckExecuted(criteria, false, maxLp)` — `isLP5Update` / `isCheckingLP5` = **false**: first-pass checks without "LP5 already refreshed" semantics.
8. **LP5 regeneration + ERG pass 2**
   - `MainExecutedClass.UpdateLP5()` (static): recomputes LP index **5** from current `TurbineDataModel` inlet / exhaust / mass (superheat-based offset, half exhaust back-pressure, etc.).
   - `ResetNozzleCounter()` between passes clears executed nozzle optimizer iteration state.
   - `ErgResultsCheckExecuted(criteria, true, maxLp)`
9. `ResetNozzleCounter()` again, then valve + Kreisl coupling + power closure
   - `ValvePointOptimize(maxLp)` — `ExecValvePointOptimizer` (executed nozzle + mass-flow iteration; see subsection below).
   - `KreislDATHandler.FillVari40()` — writes Kreisl coupling line into `TURBATURBAE1.DAT.DAT` (Vari 40) so Kreisl-aware runs stay consistent with Turba DAT.
   - `TurbaAutomation.LaunchTurba(maxLp)` — second Turba run after coupling line.
   - **Optional extra-Kreisl path** *(only when `AdditionalLoadPoint.GetInstance().CustomerLoadPoints.Count > 2`)*: may `RenameTurbaCON`, `RemoveErg`, `RefreshKreislDAT`, `FillWheelChamberPressure`, append multi-LP `KREISL.DAT` fragments via `fillAGainDat` / `fillLPAgain`, optional `UpdateDesupratorWithTurba`, then `LaunchKreisL()`.
   - **Always afterward**: read wheel chamber pressure from `TurbaOutputModel`, `KreislIntegration.RenameTurbaCON("...\Turman250\TURBATURBAE1.DAT.CON", "...\TURBA.CON")`, `KreislDATHandler.FillWheelChamberPressure(StartKreisl.filePath, "1 0", wheelChamberP)`.
   - `CheckPower(maxLp)` → `ExecPowerMatch.CheckPower` in `src/core/Checks/Exec_ERG_PowerMatch.cs` (Section 7.10.1 — `CorrectLP5Bending` and related loops).

11.6.3.1 Deeper: what each Executed step calls (mini call trees)

Same idea as **Section 11.6.1.1**, but for the executed stack and files.

A) `PowerKNN.ExecutePowerKNN(criteria)` (`src/core/HMBD/Exec_Power_KNN.cs`)

- Reads `AppSettings:ExcelFilePath` workbook sheets `PowerDB`, `PowerNormDB`, `PowerNearest`.
- Normalizes the current case (pressure, temperature, mass, exhaust pressure).
- Filters historical rows by `criteria`:
  - `BCD1120`: normalized column 8 in band **1120–1130**
  - `BCD1190`: band **1190–1210**
  - `Throttle`: column 9 text `Throttle` (and caps `k` at **2** when `k > 2`)

B) `MoveYAndSetParams()` → `FlowPathSelector.MoveYAndSetParams`

- `AddOrMoveY()` — manipulates which row in `ListPower` is marked `KNearest = "Y"` (neighbor rotation / "try next" bookkeeping; uses `MainExecutedClass.row` when higher-efficiency solve flag is on).
- `UpdateHBDParamsExecuted(updatedRow)` — `ExecHMBDConfiguration.UpdateHBDParamsExecuted` copies the selected neighbor's parameters into the HMBD / turbine model context for the rest of the run.

C) `ReferenceDATSelectorExecuted(criteria)`

- `GetFlowPathExecuted(criteria)`
  - sets `turbineDataModel.TurbineStatus = criteria`
  - `SelectExecutedFlowPath(criteria)` walks `ListPower` for the row with `KNearest == "Y"`, resolves `ProjectName` against `ExecutedDB.ExecutedProjectDB`, sets `ClosestProjectID`, `ClosestProjectName`, `DatFilePath`, returns repository `.DAT` path string.
  - `CopyRefDATFile(path)` — verifies file readiness, copies into `C:\testDir\` **, renames to ** `TURBATURBAE1.DAT.DAT` **, ** `UpdateHeaderOrderUserDate` (order / user / date line).

D) `LoadDatFile()` + `GenerateLoadPoints(maxLp)` + `PrepareDATFileExecuted(maxLp)`

- `ExecutedDATFileProcessor.LoadDatFile()` — reads `C:\testDir\TURBATURBAE1.DAT.DAT` → `turbineDataModel.DAT_DATA`.
- `ExecLoadPointGenerator.GenerateLoadPoints(maxLp)` — builds the same style of internal LP table as standard (base, backpressure variants, temperature offset, MCR-style points, etc.); may touch `KREISL.DAT` snapshot (`mainTemp`), `KreislDATHandler` RPM init when `StartKreisl.kreislKey`, deletes stray `TURBA.CON`.
- `PrepareDATFileExecuted` — `ExecutedDATFileProcessor.PrepareDatFileExecuted`
  - `LoadLP1FromDat`, `DeleteRowAfterFirstLoadPoint`, `InsertDataLineUnderFirstLPFixed`, `DeleteLoadPoints`, `InsertLoadPointsWithExactFormattingUsingMid`, `InsertDataLineUnderND`
  - `DatFileInitParamsExceptLPExecuted()` — executed variant of powertrain / nozzle / vari writes (uses `thermodynamicService.GetInletVelocity`, `getVolumetricFlow`, `PowerEfficiencyModel`, generator/gearbox/turbine update helpers — same family as standard `DatFileInitParamsExceptLP` but in `Exec_DAT_Handler.cs`).

E) `IsWheelChamberPressureValid()`

- `GetParam_RADKAMMER()` parses `RADKAMMER` from `turbineDataModel.DAT_DATA`
- Compares against `PreFeasibilityDataModel.InletPressureActualValue` / `BackpressureActualValue`:
  - invalid if `RADKAMMER < backPressure` OR `RADKAMMER > 0.8 * inletPressure`

F) `ErgResultsCheckExecuted(criteria, isLP5Update, maxLp)`

- **Called twice in the executed pipeline** (see Section 11.6.3 execution order):
  - **Pass 1**: after the first `LaunchTurba(maxLp)`, call `ErgResultsCheckExecuted(criteria, false, maxLp)`.
  - **Update LP5**: `MainExecutedClass.UpdateLP5()` rewrites LP5 (index 5) to a regenerated "stress" point.
  - **Pass 2**: call `ErgResultsCheckExecuted(criteria, true, maxLp)` to re-validate ERG gates after LP5 changed.
- Sets the appropriate static flag then dispatches:
  - `BCD1120` → `ERG_BCD1120.isCheckingLP5 = isLP5Update` → `ErgResultsCheckBCD1120(maxLp)`
    - Gate chain (order in code): exhaust volumetric → **nozzles** (`ExecutedNozzleOptimizer.RuleEngineAlgorithmForNozzles`) → thrust → delta-T / GBC / wheel / bending → `ErgResultsCheckBCD1120New`
    - Failure paths often recurse `MainExecuted("BCD1190", maxLp)` or `ResetCleanUpExecutedNearest()` then hand off.
  - `BCD1190` → `ERG_BCD1190.isLP5Update = isLP5Update` → `ErgResultsCheckBCD1190(maxLp)`
    - Gate chain: `ErgCheckExhaust1190` (exhaust curves / delta-T bands) → nozzles → thrust → delta-T / GBC / wheel / bending → `ErgResultsCheckBCD1190New`
    - Failed exhaust / neighbor exhaustion paths call `MainExecuted("BCD1190", maxLp)` again.
  - `Throttle` → `ERGResultsChecker.ERGResultsCheckThrottle()` (throttle-specific ERG gate file).

G) `MainExecutedClass.UpdateLP5()` (static)

- Uses `IThermodynamicLibrary.tsatvonp(InletPressure)` to derive a superheat offset pattern, then overwrites `LoadPointDataModel.LoadPoints[5]` fields (pressure, temp, mass, back-pressure, rpm, flags) from `TurbineDataModel`.

H) `ExecValvePointOptimizer.ValvePointOptimize(maxLp)`

- Reads `TurbaOutputModel` LP1 and LP6 `ABWEICHUNG` and LP6 nozzle group state.
- If already within **0–0.5%**, returns.
- Otherwise `AdjustNozzlePair` → `ExecutedNozzleOptimizer.UpdateNozzleSpecs` → `TurbaAutomation.LaunchTurba` → recursive `ValvePointOptimize`, or `AdjustValvePointMassFlow` loop:
  - `PrepareDatFileOnlyLPUpdate` → `TurbaAutomation.LaunchTurba` until deviation sign / step-size convergence.

I) Post-valve: `FillVari40` **→ Turba → (optional multi-custom-LP Kreisl) → CON rename → wheel pressure →** `ExecPowerMatch.CheckPower`

- `FillVari40` / `FillWheelChamberPressure` touch `C:\testDir\TURBATURBAE1.DAT.DAT` and `StartKreisl.filePath` (`KREISL.DAT`).
- `ExecPowerMatch.CheckPower`: executed power closure, including `CorrectLP5Bending` paths documented under **Section 7.10.1**.

11.6.4 Custom path — `CustomExecutedClass.Main_CustomFlowPathTest(mxlp)` in `src/Main_Custom.cs`

Execution order (the major blocks you should follow in code):

1. **Setup**
   - `fillDependencies()`, `DeleteCONFiles()`, `RefreshKreislDAT()`
   - `FillClosestTurbineEfficiency()`, `GetTurbaCON(ClosestProjectID)`, `FillInputValues()`
2. **Generate custom load points**
   - `CustomLoadPointGenerator.GenerateLoadPoints(mxlp)`
3. **Pre-feasibility decision**
   - `preFeasibilityDataModel.fillPrefeasibilityDecisionChecks()`
4. **Seed nearest custom parameters + prepare custom reference DAT**
   - `CustomDatFileHandler.GetNearestParams_Custom()`
   - `CuPunConvertor.DeleteExecutedDat()`
   - copy a baseline `10LP_TURBATURBAE1.DAT.DAT` into the custom repo and `CuFlowPathSelector.CopyRefDATFile(...)`
5. **Build the custom working DAT**
   - `CustomDATFileProcessor.PrepareDatFile(mxlp)`
   - `CustomSaxaSaxi.BCD_UPDATE(mxlp)`
6. **Optimize**
   - `RelationshipAwarePSOOptimizer.InvokeTurbineDesigner()` (relationship-aware PSO only; see **Section 8.6**)
7. **Base checks + conversions**
   - `CustomERGCheck1120.ERG_CUSTOM_BASE_CHECKS()`
   - `CuPunConvertor.TurnaConvert(mxlp)`
   - `CuPunConvertor.UpdatePunConvertor()`
8. **Custom criterion checks**
   - branch to `ErgResultsCheckBCD1120_Custom` or `ErgResultsCheckBCD1190_Custom`, with LP5 re-check
9. **Close**
   - `customPowerMatch.checkFinalTurbine()` (uses `CustomPowerMatch.CheckPower(maxLp)`; see Section 8.9)

11.6.4.1 Deeper: what each Custom step calls (mini call trees)

Below is the same "mini call tree" style as **Section 11.6.1.1** (standard) and **Section 11.6.3.1** (executed), but for the **custom** pipeline.

A) Setup and Kreisl bootstrap (`src/Main_Custom.cs`)

- `CustomExecutedClass.fillDependencies()`
  - builds `CustomExecutedClass.GlobalHost` (`CreateHostBuilder` adds `IThermodynamicLibrary`, `ILogger`, `IERGHandlerService`)
  - assigns the same host to `MainExecutedClass.GlobalHost`, `StartKreisl.GlobalHost`, `StartExec.GlobalHost`
  - fills models: `NozzleTurbaDataModel`, `ExecutedDB`, `PowerEfficiencyModel`, `PreFeasibilityDataModel`, `LoadPointDataModel`, `TurbaOutputModel`
  - seeds `TurbineDataModel.ListPower[0]` with the current case boundary conditions and `KNearest = NoOfExecuted`
- `CustomExecutedClass.DeleteCONFiles()` deletes `*.CON` and `*.ERG` under `C:\testDir` (and `C:\testDir\Turman250` if present)
- `KreislDATHandler.RefreshKreislDAT()` selects and copies the correct Kreisl template into `C:\testDir\KREISL.DAT`
- `thermodynamicService.FillClosestTurbineEfficiency()` primes "closest efficiency" lookup for the initial Kreisl/HBD seed

- `CustomHMBDConfiguration.GetTurbaCON(ClosestProjectID)` pulls a starting Turba CON context for the nearest executed project
- `CustomExecutedClass.FillInputValues()` writes inlet/exhaust and cycle extras into `KREISL.DAT` (deaerator / PST / dump-condenser variants)
  - note: the method is called twice in `Main_CustomFlowPathTest` with a `RefreshKreislDAT()` between them (a "refresh then refill" pattern)

B) Generate custom load points (`src/core/HMBD/Custom_LoadPointGenerator.cs`)

- `CustomLoadPointGenerator.GenerateLoadPoints(mxlp)`
  - builds `LoadPointDataModel.LoadPoints[...]` for the custom run
  - is followed by another `customHMBDConfiguration.HBDSetDefaultCustomerParamsKreisL()` call in `Main_CustomFlowPathTest`

C) Decide which custom criterion gates apply

- `preFeasibilityDataModel.fillPrefeasibilityDecisionChecks()`
- `Decision == TRUE` ⇒ treat as **BCD1120 custom** checks
- `Decision_2 == TRUE` ⇒ treat as **BCD1190 custom** checks

D) Seed custom "nearest params" and create a working custom DAT

- `CustomDatFileHandler.GetNearestParams_Custom()` (`src/core/Handlers/Custom_DAT_Handler.cs`)
  - pulls a nearest custom seed (geometry/starting nozzle parameters) into runtime models used later by PSO and custom DAT writing
- `CuPunConvertor.DeleteExecutedDat()` (`src/core/HMBD/Cu_Pun_Convertor.cs`)
  - clears out executed artifacts so the custom path starts from its own baseline
- baseline DAT copy:
  - copies `AppContext.BaseDirectory\\10LP_TURBATURBAE1.DAT.DAT` into `C:\testDir\projects_repository\custom_flowPaths\`
  - `CuFlowPathSelector.CopyRefDATFile(...)` (`src/core/HMBD/Cu_Ref_DAT_Selector.cs`) normalizes it into `C:\testDir\TURBATURBAE1.DAT.DAT`

E) Build the custom DAT + apply SAXA/SAXI initial updates

- `CustomDATFileProcessor.PrepareDatFile(mxlp)` (`src/core/Handlers/Custom_DAT_Handler.cs`)
  - writes LP blocks + base parameters into `TURBATURBAE1.DAT.DAT` (custom variant of the standard DAT preparation flow)
- `CustomSaxaSaxi.BCD_UPDATE(mxlp)` (`src/core/Checks/Cu_Saxa_Saxi.cs` / `src/core/Checks/SAXA_SAXI`)
  - applies BCD-specific SAXA/SAXI updates before optimization

F) PSO-based optimization (`src/core/Optimizers/Cu_PSOFlowPathOptimizerNozzle.cs`)

- `RelationshipAwarePSOOptimizer.InvokeTurbineDesigner()`
  - entry point runs **only** `PSOLoop()` (relationship-aware particle swarm—see **Section 8.6**)
  - each candidate updates the DAT soft checks, runs Turba via `CuTurbaAutomation`, and scores feasibility with `PenaltyScoreCalculator`

G) Conversion and custom Turba run

- `CustomERGCheck1120.ERG_CUSTOM_BASE_CHECKS()` (`src/core/Checks/Cu_ERG_BCD_1120.cs`)
  - custom "sanity gate" checks before the full criterion chain
- `CuPunConvertor.TurnaConvert(mxlp)` then `CuPunConvertor.UpdatePunConvertor()`
  - converts / rewrites steam-path related data before continuing
- `CuTurbaAutomation.LaunchTurba(mxlp)` (`src/core/Turba/Cu_TurbaConfig.cs`)
  - runs Turba, waits for `TURBA_FLAG.bin`, loads ERG into `TurbaOutputModel`

H) Two-pass custom ERG checks (LP5 regen + re-check), then valve optimization

- **Pass 1** (LP5 not yet regenerated):
  - if `Decision==TRUE`: `CustomERGCheck1120.isCheckingLP5=false` → `ErgResultsCheckBCD1120_Custom(mxlp)` (`src/core/Checks/Cu_ERG_BCD_1120.cs`)
  - else if `Decision_2==TRUE`: `CustomERGCheck1190.isCheckingLP5=false` → `ErgResultsCheckBCD1190_Custom(mxlp)` (`src/core/Checks/Cu_ERG_BCD_1190.cs`)
- `ResetNozzleCounter()` resets `CustomNozzleOptimizer` iteration state
- `UpdateLP5(mxlp)` (static in `src/Main_Custom.cs`)
  - recomputes LP5 (index 5) as the regenerated "stress" point and calls `CustomDATFileProcessor.PrepareDatFileOnlyLPUpdate(maxlp)` to rewrite only LP blocks
- **Pass 2** (validate again with regenerated LP5):
  - repeats the same checker, but with `isCheckingLP5=true`
- `CustomValvePointOptimizer.ValvePointOptimize(mxlp)` (`src/core/Optimizers/Cu_ERG_ValvePointOptimizer.cs`)
  - same structure as executed/standard: adjust nozzle pairs or adjust LP6 mass-flow, re-run Turba until the valve-point deviation converges

I) Close: Vari40 + Turba + Kreisl coupling + custom power match + final fixes

- `KreislDATHandler.FillVari40()` then `CuTurbaAutomation.LaunchTurba(mxlp)`
- `KreislIntegration.RenameTurbaCON("...\\Turman250\\TURBATURBAE1.DAT.CON", "...\\TURBA.CON")`
- if `AdditionalLoadPoint.CustomerLoadPoints.Count > 2`:
  - performs the same "merge extra LPs back into Kreisl" block as executed (rebuild `KREISL.DAT` with `fillAGainDat` / `fillLPAgain`, optional desuperheater updates, then `LaunchKreisL()`)
- always: `FillWheelChamberPressure(...)` back into Kreisl-side DAT (`KREISL.DAT`)
- `CustomPowerMatch.CheckPower(mxlp)` (`src/core/Checks/Cu_ERG_PowerMatch.cs`)
  - custom power closure (includes `CorrectLP5Bending()` concept as documented in **Section 8.9**)
- post-closure cleanup/fixes:
  - `customERGCheck1120.ERG_CUSTOM_TURNA_CHECKS()`
  - `customSaxaSaxi.SAXA_FIX()`
  - final Turba run + `customPowerMatch.checkFinalTurbine()`

11.6.5 Additional load points — `CustomLoadPointHandler.cxLP_mainKreisl(customerLPList)` in `src/AdditionalLoadPoints.cs`

This one is easiest to follow in the same three phases used in **Section 9.1.1**:

1. **Kreisl-side LP preparation**
   - `checkingPartLoadExist(customerLPList)` → sets `PowerGeneration` for part-load rows
   - snapshot `initList` + `lpNumberToIndexMap`
   - `DeleteCONFiles()` + `fillCustomerLoadPointList(customerLPList)`
   - `RefreshKreislDAT()` + `cxLP_GetLPcount()` + `fillLPINDat()`
   - back-fill missing values from `KREISL.ERG` and compute `VolFlow`
   - `SortCustomerLoadPointsByVol()`
   - closed-cycle extras: add capacity LP, PST default, bump `cxLP_RngStop`
2. **Standard-mirror Turba stabilization**
   - `ReferenceDATSelector(cxLP_RngStop + 10)`
   - `cxLP_GenerateLoadPoints("Recal")` + `GenerateLoadPoints()`
   - `prepareDATFile(cxLP_RngStop + 10)`
   - `LaunchTurba(...)` + `ergResultsCheck(...)`
   - `UpdateLP5()` + second `ergResultsCheck(...)`
   - `ValvePointOptimize(...)`
   - `FillVari40()` + `RefreshKreislDAT()` + wheel chamber pressure write-back
3. **Merge extra customer LPs back into Kreisl**
   - loop customer LPs:
     - `i==1`: `fillAGainDat(index, initList)`
     - else: choose missing dimension and call `fillLPAgain(index,"Pr/T/M/P/E",count,initList)`

- `File.WriteAllText("C:\\testDir\\KREISL.DAT", MainTemp)`
- if closed/PST: `UpdateDesupratorWithTurba(...)`
- `LaunchKreisL()` then `CheckPower(10 + cxLP_RngStop)`

---

12) Notes

- If diagrams show as a `mermaid` **code block** instead of a picture, your editor preview is not rendering Mermaid (re-enable a **Markdown Mermaid** extension, or view this file on GitHub). A recent Cursor/VS Code or extension update can turn that off.
- **PDF export (Mermaid renders as diagrams):** from folder `my-project/docs`, run `npm install` then `npm run pdf`. This writes `README.pdf` next to this file (loads Mermaid from a CDN, then prints via headless Chromium). First run downloads Puppeteer's browser; Internet access is required.
- This README is code-logic first and intentionally flow-oriented.
- Keep terminology as in code where possible (`DumpCondensor`, `DeaeratorOutletTemp`, `IsPRVTemplate`) to avoid mismatch.
- Extend each section with diagrams and method-level call mapping as documentation evolves.